

Salesforce Project Documentation

Re Plastic Innovations CRM

Contents

1 Project Overview	2
2 Objectives and Scope	2
2.1 Primary Business Objectives	2
2.2 Project Scope	2
3 Phase 1: Requirement Analysis & Planning	3
3.1 Understanding Business Requirements	3
3.2 Data Model Architecture	4
3.3 Object Tabs and Navigation Planning	10
4 Phase 2: Salesforce Development - Backend & Configurations	11
4.1 Declarative Automation: Schedule-Triggered Flow	11
4.2 Low Stock Follow-Up - End-to-End Verification	13
4.3 Programmatic Automation: Apex & Triggers	14
4.4 Inventory Validation - UI Testing	17
4.5 Stock Reduction after a Valid Order	17
4.6 Apex Trigger Definitions	18
4.7 Restock Approval and Email Notification	19
4.8 Formula Field - Stock Low On Product	21
4.9 Validation Rules	22
4.10 Roles, Profiles and User Creation	24
4.11 Object-Level and Record-Level Access	27
5 Phase 3: UI/UX Development & Customization	30
5.1 Lightning App & Tab Setup	30
5.2 Record Pages Used During Testing	31
6 Phase 4: Data Handling, Testing & Quality Assurance	31
6.1 Test Data Preparation	31
6.2 Functional Testing Matrix	32
6.3 Apex Test Coverage Scope	32
7 Phase 5: Deployment, Documentation & Maintenance	36
7.1 Deployment Readiness	36
7.2 System Monitoring & Troubleshooting	36
8 Implementation Architecture Summary	38
8.1 Completed Feature Summary	38
8.2 Future Enhancements	39
9 Conclusion	39

1 Project Overview

Re Plastic Innovations, also referred to as Replastix, is a Salesforce application for managing plastic waste collection, recycling centers, recycled products, customer orders and restock requests. The project implements an eco-friendly operational system to support the recycling of plastic waste into sustainable products, promoting a circular economy and reducing environmental impact.

The solution centralizes the information needed to follow waste collections and maintain recycled product inventory. It combines custom Salesforce objects, lookup relationships, a Lightning application, role and profile configuration, sharing rules, formula fields, validation rules, a scheduled Flow and Apex inventory automation.

When stock is sufficient for a new order, Apex reduces the related product inventory. When stock is insufficient, the system creates a Pending restock request for the shortage. A transition to Approved adds the requested stock and invokes a warehouse notification. A separate daily Flow identifies low-stock products and creates follow-up Tasks. These features support administrative coordination; the application does not itself perform physical recycling or calculate environmental savings.

2 Objectives and Scope

2.1 Primary Business Objectives

- Centralize recycling operations: maintain dedicated records for Plastic Waste, Recycling Centers, Recycled Products, Orders and Restock Requests.
- Track collection details: record waste weight, plastic type, collection date, status, collection location and assigned recycling center.
- Maintain inventory visibility: store stock levels, minimum stock thresholds and unit prices, and display a low-stock message when appropriate.
- Automate inventory handling: reduce available stock for a sufficient order and create a Pending request for an order shortage.
- Support warehouse follow-up: increase stock on restock approval, notify warehouse users and schedule daily low-stock Tasks.
- Apply data and access controls: configure roles, profiles, sample users, sharing rules and validation rules, and test the Apex logic.

2.2 Project Scope

In-Scope: five custom objects and their fields, lookup relationships, custom tabs, a Lightning app, three roles, three custom profiles, three sample-user configurations, organization-wide defaults and sharing rules, a stock formula, two validation rules, a schedule-triggered Flow, InventoryManager, two Apex triggers, EmailNotificationHelper and InventoryManagerTest.

Completion boundary: the learner reported 100 percent Apex coverage. The remaining Platform 3 permissions for four custom objects were explicitly deferred. Scheduled Task creation, actual approval-email delivery and complete user-access behavior still require final evidence. Production deployment, Batch Apex, Scheduled Apex classes, dashboards, integrations and bulk migration were not performed.

3 Phase 1: Requirement Analysis & Planning

3.1 Understanding Business Requirements

The SkillWallet requirements were divided into operational records and automation. Waste records hold collection and processing details; Recycling Center records hold locations and capacity; Recycled Product records hold stock and prices; Orders connect customers to products; and Restock Requests record additional units required.

A threshold alert and an order shortage are different conditions. The formula and daily Flow compare Stock Level against Threshold. The Apex order handler compares Stock Level against the order Quantity. The first condition supports follow-up visibility; the second controls the inventory deduction or shortage request.

The SkillWallet activities were translated into the following implementation areas. Planned story durations were learning estimates, not measured project effort.

Requirement	Salesforce implementation	Outcome
Plastic waste tracking	Plastic Waste object and center lookup	Collection details and processing status are stored together.
Center management	Recycling Center object	Location and processing capacity can be maintained.
Sustainable product inventory	Recycled Product object and formula	Stock, threshold and price are visible.
Customer ordering	Custom Order object and Apex trigger	Available stock is reduced after a new order.
Shortage handling	Restock Request object and approval trigger	Shortfall is requested; approval adds inventory.
Operational follow-up	Daily scheduled Flow and Task	Low stock can prompt a high priority follow-up.
Access administration	Roles, profiles, users and sharing rules	Object access and selected record sharing are configured.
Quality assurance	Validation rules and InventoryManagerTest	Invalid values are checked and core stock logic is tested.

Table 1: Business Requirements and Salesforce Implementation

Supplied activity	Report location	Planned duration
Objects, relationships and repeated key field sections	Section 3.2	Not supplied
Tabs and Lightning app	Section 5.1	Not supplied
Roles, profiles, users and sharing	Sections 4.10 and 4.11	Platform 1 modification: 40 minutes
Stock Low On Product formula	Section 4.8	30 minutes
Validation rules	Section 4.9	Not supplied
Flow Builder	Sections 4.1 and 4.2	1 hour 59 minutes
Apex, triggers, email and tests	Sections 4.3 to 4.7 and 6.3	Apex activity: 1 hour 58 minutes

Table 2: SkillWallet Activity Mapping and Planned Story Durations

Sequence: establish the schema first, expose it in the app, configure access, add declarative rules, add Apex automation, then test and document. Repeated field instructions were treated as verification steps, not a reason to create duplicate fields.

3.2 Data Model Architecture

Each main business entity is independently stored in a custom object. The relationships are lookups, not master-detail relationships. Account and Task are standard Salesforce objects used by the solution.

Business object	Exact object API name
Plastic Waste	Re_Plastic_Innovations_Plastic_Waste__c
Recycling Center	Re_Plastic_Innovations_Recycling_Center__c
Recycled Product	Re_Plastic_Innovations_Recycled_Product__c
Order	Re_Plastic_Innovations_Order__c
Restock Request	Re_Plastic_Innovations_Restock_Request__c

Table 3: Custom Objects and Exact API Names

Lookup architecture

Child object and field	Related object	Business meaning
Plastic Waste Recycling_Center__c	Recycling Center	Center assigned to a waste collection record.
Order Account__c	Account	Customer placing the order.
Order Product__c	Recycled Product	Product selected on a customer order.
Restock Request Product__c	Recycled Product	Product for which additional stock is requested.

Table 4: Lookup Relationships and Business Meaning

One center can be referenced by many waste records, and one product can be referenced by many orders and restock requests. Each child lookup stores one related record ID. The project does not define a direct relationship from collected waste to an individual manufactured product.

Field creation pattern

1. Select the object in Object Manager. Open Fields & Relationships and click New.
2. Select the data type, click Next, and enter the label, API field name and type-specific settings.
3. For a lookup, choose the correct related object before entering the field label. For a picklist, enter one value per line.
4. Review field-level security. Keep the field visible to the configuring profile; grant other profiles only the access required for their role.
5. Include the field on the appropriate page layout. Click Save or Save & New. Verify the saved field API name.

Number settings in this report distinguish Salesforce UI Length from Decimal Places. For the decimal fields corrected in the walkthrough, Length 16 plus Decimal Places 2 satisfies the combined 18-digit limit shown by Salesforce.

Custom Object Creation

The work was performed in Salesforce Developer Edition using Setup, Object Manager, App Manager, Flow Builder and Developer Console. A System Administrator account was used for configuration. Salesforce Platform licenses were selected for the sample project users where available.

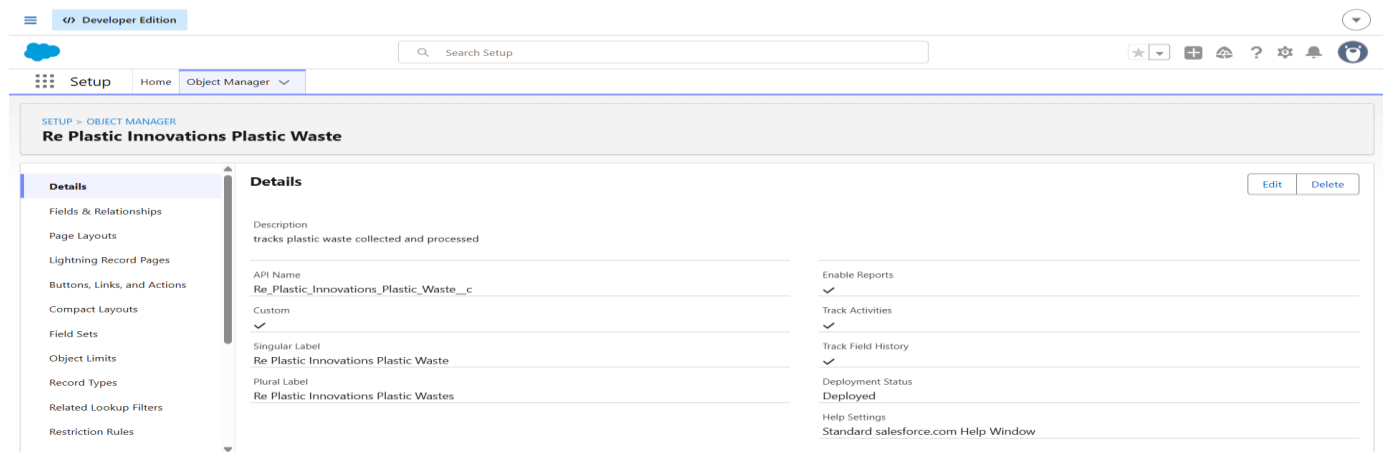
Common procedure for all five objects

1. Open the gear menu and select Setup. Open Object Manager, then Create and Custom Object.
2. Enter the object Label, Plural Label and Object Name. Salesforce adds the __c suffix to the saved custom object API name.
3. Set the standard Record Name label and data type using the table below. For auto-number records, enter the display format and starting number 1.
4. Use the project options for reports and activities. If field history tracking is enabled, selecting individual fields for history is a separate configuration step.
5. Keep the object deployed for the learning app and save. If the custom tab wizard opens, choose a tab style and finish it.
6. Open Fields & Relationships and add the fields specified in Section 3.2. Check existing definitions before using New.

Object	Record Name type	Naming format used in procedure
Plastic Waste	Auto Number	PW-{0000}
Recycling Center	Text	Recycling Center Name
Recycled Product	Text	Recycled Product Name
Order	Auto Number	Order Number; ORD-{0000}
Restock Request	Auto Number	Restock Request Number; RR-{0000}

Table 5: Record Names and Auto Number Formats

The Name field is a built-in record identifier. It is not an extra custom field and must not be deleted. Created By, Last Modified By and Owner are also normal Salesforce fields. Labels may differ from API names without changing the relationship used by code.



Plastic Waste Object and Fields

Object: Re_Plastic_Innovations_Plastic_Waste__c. Purpose: track plastic waste collected and processed, with its assigned recycling center.

Field API name	Data type or settings	Description
Name	Auto Number	Unique waste record identifier.
Weight__c	Number Length 16; decimals 2	Weight of collected plastic waste in kg.
Type__c	Picklist	PET, HDPE, PVC, LDPE, PP, PS, Other.
Collection_Date__c	Date	Date on which waste was collected.
Status__c	Picklist	Collected, Processing, Recycled.
Recycling_Center__c	Lookup to Recycling Center	Assigned recycling center.
Location__c	Geolocation Decimal degrees; 6 decimals	Waste collection latitude and longitude.

Table 6: Plastic Waste Field Specifications

Configuration details

1. Create Weight as Number. Use Length 16 and Decimal Places 2 for the corrected decimal specification. Save and create the next field.
2. Create Type as a Picklist with PET, HDPE, PVC, LDPE, PP, PS and Other on separate lines. The guide phrase "etc." is descriptive, not a literal picklist value.
3. Create Collection Date as Date, and Status as a Picklist with Collected, Processing and Recycled.
4. Create Recycling Center as a Lookup Relationship to the custom Re Plastic Innovations Recycling Center object.

5. Use Geolocation for Location. If an earlier text location exists, retain it as a legacy address field before adding the coordinate field; do not erase stored address data.

The source table described Name as “Text (Auto Number)”. These are different record-name types; Auto Number is the intended unique-ID configuration. The later Weight instruction with Length 10 was superseded by the corrected decimal field specification.

The screenshot shows the Salesforce Setup interface, specifically the 'Object Manager' for 'Re Plastic Innovations Plastic Waste'. The 'Fields & Relationships' tab is selected, showing a list of 9 fields. The fields are: Collection Date (Date), Created By (Lookup(User)), Last Modified By (Lookup(User)), Location (Text(255)), Owner (Lookup(User,Group)), Re Plastic Innovations Plastic Waste Name (Auto Number), Status (Picklist), Type (Picklist), and Weight (Number(16, 2)). The 'Name' field is highlighted as the primary field.

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Collection Date	Collection_Date__c	Date		
Created By	CreatedById	Lookup(User)		
Last Modified By	LastModifiedById	Lookup(User)		
Location	Location__c	Text(255)		
Owner	OwnerId	Lookup(User,Group)		
Re Plastic Innovations Plastic Waste Name	Name	Auto Number		
Status	Status__c	Picklist		
Type	Type__c	Picklist		
Weight	Weight__c	Number(16, 2)		

Recycling Center Object and Fields

Object: Re_Plastic_Innovations_Recycling_Center__c. Purpose: maintain recycling centers, their coordinates and their maximum waste processing capacity.

Field API name	Data type or settings	Description
Name	Text	Recycling Center Name.
Location__c	Geolocation Decimal degrees; 6 decimals	Center latitude and longitude.
Capacity__c	Number Length 16; decimals 2	Maximum waste processing capacity in kg; the time period is not defined in the supplied scope.

Table 7: Recycling Center Field Specifications

Configuration procedure

1. Open the Recycling Center object and confirm the Record Name is a Text field. “Recycling Center Name” is an appropriate label.
2. In Fields & Relationships, create Location using Geolocation, with decimal degrees and six decimal places.
3. Create Capacity as Number, using Length 16 and Decimal Places 2.
4. Review visibility and page layout placement, save, and check the final field list.
5. Confirm the Plastic Waste Recycling Center lookup points to this object, not to an unrelated production object.

During the walkthrough, the initial text location was retained under the legacy address naming approach before a Geolocation field was introduced. Legacy fields are not part of the required final schema; verify their exact saved names before including them in an org export.

Recycled Product Object and Fields

Object: Re_Plastic_Innovations_Recycled_Product__c. Purpose: store sustainable recycled product names, current inventory, minimum stock thresholds and selling prices.

Developer Edition

Setup Home Object Manager

Re Plastic Innovations Recycling Center

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Restriction Rules

Details

Description

API Name
Re_Plastic_Innovations_Recycling_Center__c

Custom
✓

Singular Label
Re Plastic Innovations Recycling Center

Plural Label
Re Plastic Innovations Recycling Centers

Enable Reports
✓

Track Activities
✓

Track Field History
✓

Deployment Status
Deployed

Help Settings
Standard salesforce.com Help Window

Edit Delete

Developer Edition

Setup Home Object Manager

Re Plastic Innovations Recycling Center

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Restriction Rules

Fields & Relationships

6 Items, Sorted by Field Label

Quick Find

New Deleted Fields Field Dependencies Set History Tracking

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Capacity	Capacity__c	Number(16, 2)		
Created By	CreatedById	Lookup(User)		
Last Modified By	LastModifiedById	Lookup(User)		
Location	Location__c	Text(255)		
Owner	OwnerId	Lookup(User,Group)		✓
Recycling Center Name	Name	Text(80)		✓

Field API name	Data type or settings	Description
Name	Text	Recycled product name; provide it when creating products in Apex tests.
Stock_Level__c	Number Length 16; decimals 0	Current units available.
Threshold__c	Number Length 16; decimals 0	Minimum stock level used for the low-stock comparison.
Price__c	Currency Length 16; decimals 2	Price per unit in the org currency.
Stock_Low_On_Product__c	Formula returning Text	Low Stock - Restock Needed, or Sufficient Stock; see Section 4.8.

Table 8: Recycled Product Field Specifications

Configuration procedure

1. Confirm Name is Text. Create Stock Level and Threshold as Number fields with zero decimal places.
2. Create Price as Currency and allow two decimal places.
3. Add the fields to the page layout and confirm that the configuring user can see them.

4. Create the Stock Low On Product formula after the base fields exist. Its source fields must use the exact API names shown above.
5. Use this custom object for Order and Restock Request lookups. Do not select the standard Salesforce Product object.

The formula and the scheduled Flow compare stock against Threshold. The Apex order logic instead compares stock against the quantity on the order. A low threshold result alone does not create a Restock Request in the current Apex design.

SETUP > OBJECT MANAGER
Re Plastic Innovations Recycled Product

Details

Description
Stores recycled products and stock levels

API Name
Re_Plastic_Innovations_Recycled_Product__c

Custom
✓

Singular Label
Re Plastic Innovations Recycled Product

Plural Label
Re Plastic Innovations Recycled Products

Enable Reports
✓

Track Activities
✓

Track Field History
✓

Deployment Status
Deployed

Help Settings
Standard salesforce.com Help Window

Edit Delete

SETUP > OBJECT MANAGER
Re Plastic Innovations Recycled Product

Fields & Relationships
 7 Items, Sorted by Field Label

Quick Find New Deleted Fields Field Dependencies Set History Tracking

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Price	Price__c	Currency(16, 2)		
Recycled Product Name	Name	Text(80)		✓
Stock Level	Stock_Level__c	Number(16, 0)		
Threshold	Threshold__c	Number(16, 0)		

Order Object and Fields

Object: Re_Plastic_Innovations_Order__c. Purpose: capture the customer, product, quantity and expected delivery date of an order for recycled products.

Actual field API name	Data type or settings	Description
Name	Auto Number	Order Number; order identifier.
Account__c	Lookup to Account	Customer placing the order.
Product__c	Lookup to Recycled Product	Ordered custom recycled product.
Quantity__c	Number Length 16; decimals 0	Number of units ordered.
Delivery_Date__c	Date	Expected delivery date.

Table 9: Order Field Specifications

Specification to implementation mapping

Name in supplied guide	Name in the observed Order object	Action taken
Customer__c	Account__c	Keep the existing Account lookup.
Recycled_Product__c	Product__c	Update Apex to use Product__c.
Product mentioned in initial field list	Product__c	Use this saved API name consistently.

Table 10: Guide Names and Actual Order API Names

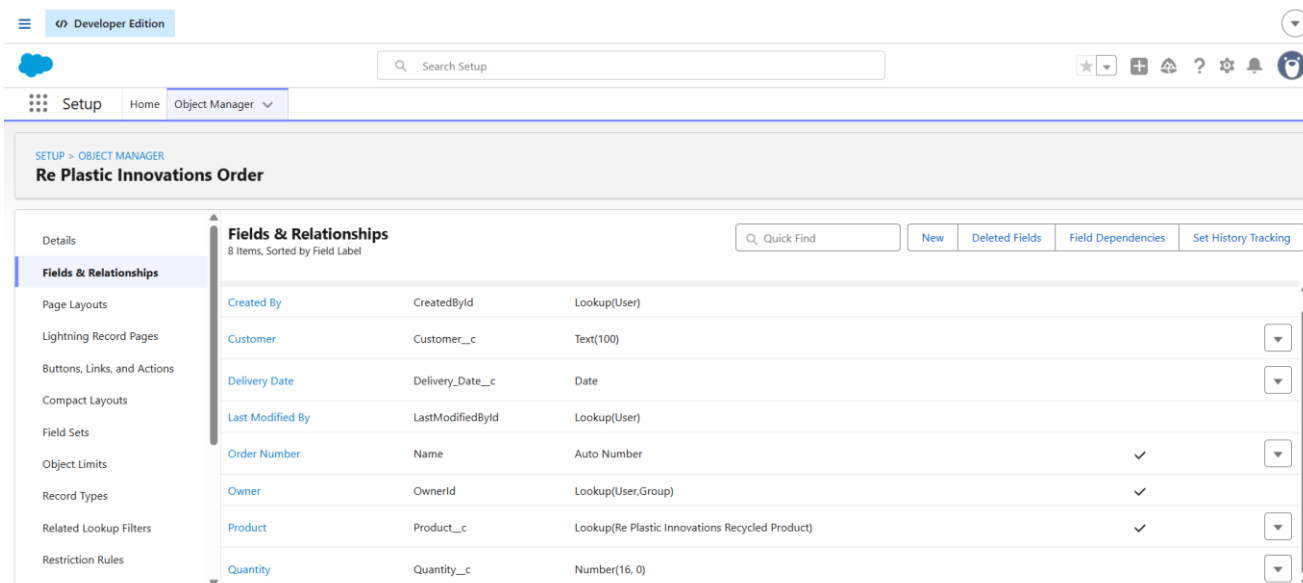
1. Verify Account is a lookup to the standard Account object and Product is a lookup to the custom Recycled Product object.
2. Create or verify Quantity and Delivery Date, then include them on the layout.
3. Retain the existing field names shown in the screenshot. Do not create a duplicate Recycled_Product__c field just to make the original sample compile.
4. Add the Check_Quantity_Not_Zero validation rule described in Section 4.9.

The supplied screenshot showed eight fields including standard audit and ownership fields. Those additional standard fields are expected and do not indicate a schema error.

The screenshot shows the Salesforce Setup interface in Developer Edition. The navigation menu on the left includes Setup, Home, and Object Manager. The main content area is titled 'Re Plastic Innovations Order' under the 'OBJECT MANAGER' section. The 'Details' tab is selected, showing the following information:

- Description:** Handles customer orders for recycled products
- API Name:** Re_Plastic_Innovations_Order__c
- Custom:** ☒
- Singular Label:** Re Plastic Innovations Order
- Plural Label:** Re Plastic Innovations Orders
- Enable Reports:** ☒
- Track Activities:** ☒
- Track Field History:** ☒
- Deployment Status:** Deployed
- Help Settings:** Standard salesforce.com Help Window

Buttons for 'Edit' and 'Delete' are visible in the top right corner of the details section.



Restock Request Object and Fields

Object: Re_Plastic_Innovations_Restock_Request__c. Purpose: track products needing additional stock, the requested quantity and the request status.

Field API name	Data type or settings	Description
Name	Auto Number	Restock request identifier; RR-{0000} in the creation procedure.
Product__c	Lookup to Recycled Product	Product to restock.
Requested_Quantity__c	Number Length 16; decimals 0	Additional units requested.
Status__c	Picklist	Pending, Approved, Completed. Pending is the initial request status.

Table 11: Restock Request Field Specifications

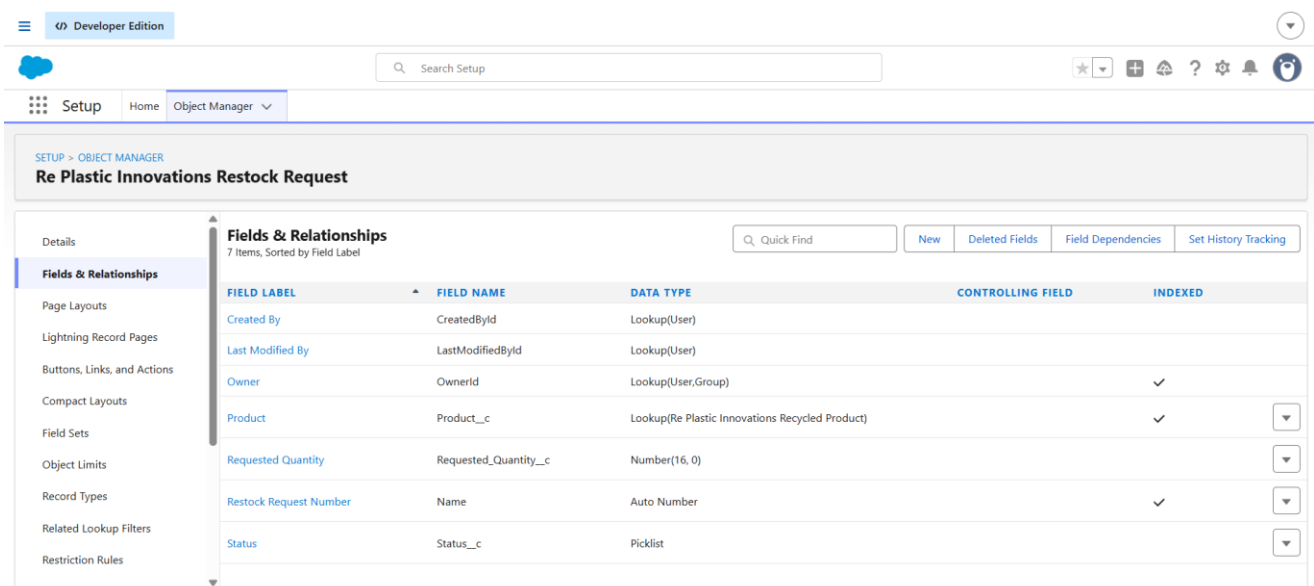
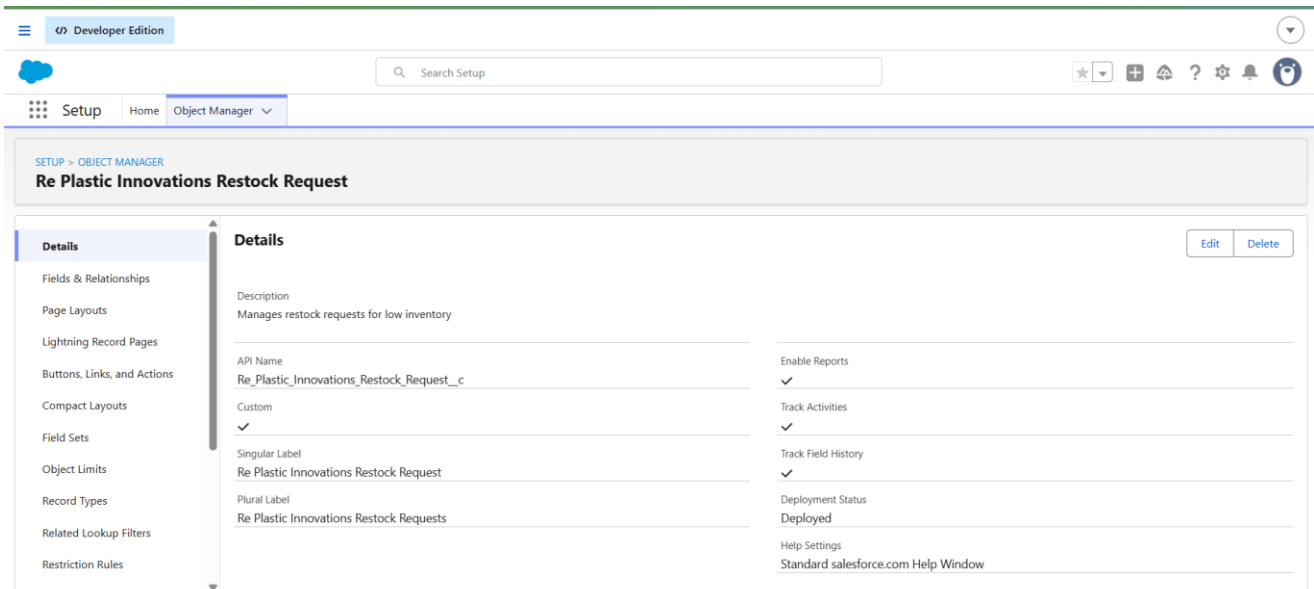
Configuration procedure

1. Create the object with an Auto Number record name. Do not set Name manually when Apex creates a restock request.
2. Create Product as a Lookup Relationship to the custom Recycled Product object.
3. Create Requested Quantity as Number with zero decimal places.
4. Configure Status with Pending, Approved and Completed; use Pending for a new request.
5. If the initial setup contains other status values, deactivate unwanted values after checking existing records instead of deleting the field.
6. Review the field list and layout before adding the Apex approval trigger.

Approved is used by the Apex trigger to add the requested quantity to product stock. Completed is available as a status value, but the supplied implementation does not automatically set it and does not implement a formal Salesforce Approval Process.

3.3 Object Tabs and Navigation Planning

1. In Setup, type Tabs in Quick Find and open Tabs.
2. Under Custom Object Tabs, select New only if a tab for the required project object does not already exist.
3. Choose the custom object and a tab style. Click Next.
4. Review the Add to Profiles page and continue with the project defaults.
5. Review Add to Custom Apps and save. Repeat only for the objects without an existing tab.

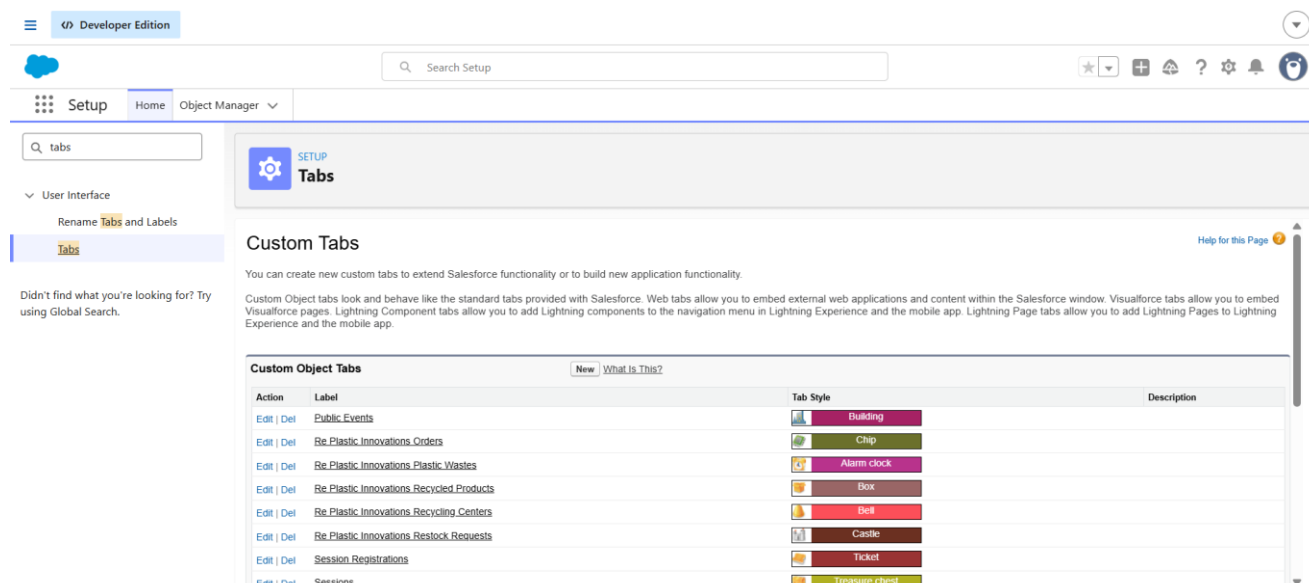


6. When the walkthrough reached this section, the five tabs already existed. They were retained instead of duplicated.

Required project tab	Object to select
Plastic Wastes	Re Plastic Innovations Plastic Waste
Recycling Centers	Re Plastic Innovations Recycling Center
Recycled Products	Re Plastic Innovations Recycled Product
Orders	Re Plastic Innovations Order
Restock Requests	Re Plastic Innovations Restock Request

Table 12: Project Custom Tabs and Related Objects

The tab section supplied with the project had a Recycling Center heading but selected Plastic Waste in the body. The same wizard applies to either object; choose the object for the particular tab being checked. The Order and Product tabs must be the project custom objects, not similarly named standard objects.



4 Phase 2: Salesforce Development - Backend & Configurations

4.1 Declarative Automation: Schedule-Triggered Flow

The Stock Level Is Low Flow evaluates recycled products daily. A Decision compares each product Stock Level with its Threshold. The low-stock outcome creates a Task assigned to the product owner; the Default Outcome ends without creating a Task.

Start and schedule

1. In Setup, search Flows and click New Flow.
2. Choose Start From Scratch when prompted. Select Schedule-Triggered Flow and create it.
3. Open Start and set the date, time and frequency. The supplied requirement is a daily run at 6:00 AM.
4. Select Re_Plastic_Innovations_Recycled_Product__c as the object. The walkthrough used no entry filters, with the low-stock check in a Decision element.
5. For a new or repeated scheduling test, select a future execution time. The saved canvas screenshot showed 12 September 2026 at 6:00 AM, Daily; that historical date is not a reusable future test schedule.

Start setting	Configured value
Flow type	Schedule-Triggered Flow
Flow label	Stock Level Is Low
Frequency and time	Daily, 6:00 AM
Object	Re_Plastic_Innovations_Recycled_Product__c
Time zone	Organization default time zone; verify it in the org. [R2]

Table 13: Daily Flow Start Configuration

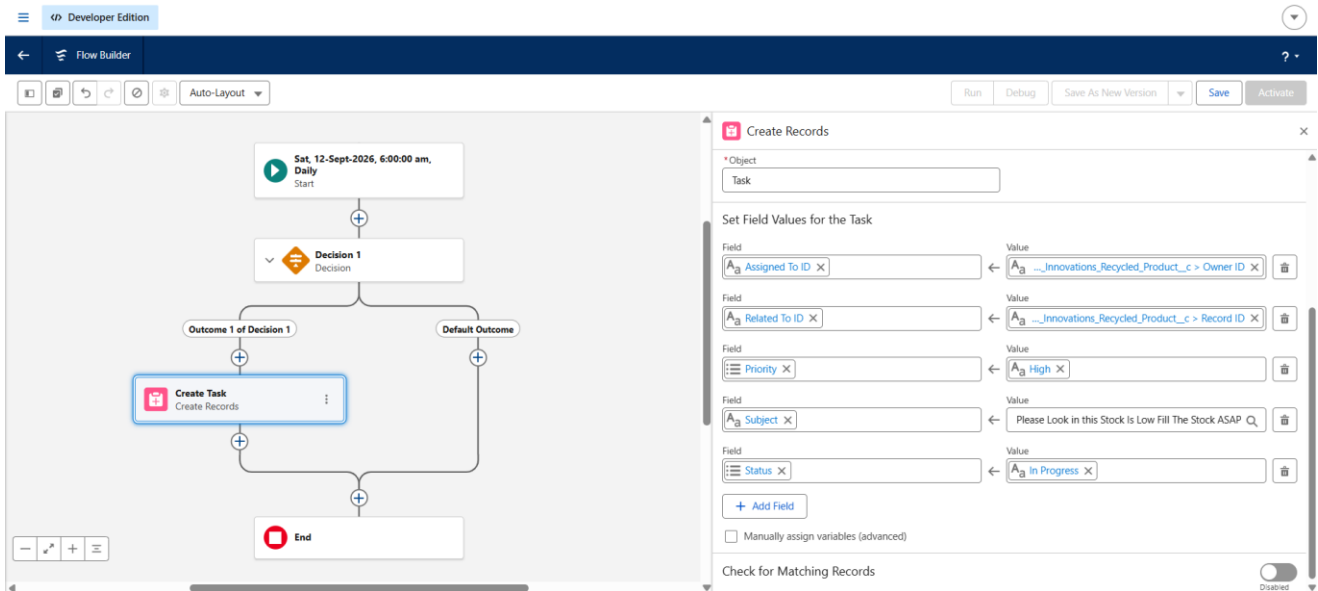
Decision element

Click the plus under Start and select Decision. Use Label Decision 1 and an outcome labeled Outcome 1 of Decision 1. Set the condition as follows:

Resource	Operator	Value resource
\$Record.Stock_Level__c	Less Than	\$Record.Threshold__c

Table 14: Low Stock Decision Condition

The matching outcome proceeds to Create Task. The Default Outcome ends without creating a Task. The newer card-based canvas can join both branches into one End; that layout is equivalent to separate End nodes when the branch connections are the same.



1. Under Outcome 1 of Decision 1, click the plus and select Create Records.
2. Enter Create Task as Label. Choose to create one record and set field values manually.
3. Select Task as the object and enter the field mappings below.
4. Leave the Default Outcome connected to End. Confirm Create Task exists only on the low-stock branch.
5. Save the Flow as Stock Level Is Low, review any errors, then Activate the saved version.
6. Capture the whole canvas and the active status. A successful save or an active canvas does not alone prove that a scheduled run created a Task.

Task field label	API field	Value
Assigned To ID	OwnerId	\$Record.OwnerId
Related To ID	WhatId	\$Record.Id
Priority	Priority	High
Subject	Subject	Please Look in this Stock Is Low Fill The Stock ASAP
Status	Status	In Progress

Table 15: Task Field Mappings

The Related To mapping associates the Task with the recycled product, so the custom object must allow activities. No due date was configured in the supplied Task mapping.

4.2 Low Stock Follow-Up - End-to-End Verification

The Flow configuration and branch layout were completed during the walkthrough. The actual creation of a Task at the scheduled time was not separately confirmed, so this test remains an evidence item rather than a claimed runtime pass.

1. Select a product with Stock Level lower than Threshold and record its owner and record name.
2. Use a future scheduled test time, taking the organization time zone into account. Save and activate the intended Flow version.
3. After the scheduled execution, inspect the product Activity area or the Tasks list, including Upcoming & Overdue where available.
4. Verify the Task subject, related product, assigned owner, Priority High and Status In Progress. Record the run time and capture the actual Task.
5. Restore the intended daily 6:00 AM schedule after a temporary scheduling test.

The Flow runs on its schedule, not immediately when a product is saved. No due date or duplicate-Task check was configured; a product that remains low can receive another Task on a later daily run. [R2]

4.3 Programmatic Automation: Apex & Triggers

InventoryManager and the order trigger

InventoryManager separates stock business logic from the triggers. UpdateStockAfterOrder runs after a new custom Order is inserted and passes Trigger.new to processOrderStock. The saved lookup field on Order is Product__c.

Order condition	Implemented action	Stock result
Product exists and available stock $\geq$ ordered quantity	Reduce stock by the order quantity.	Remaining stock is saved.
Product exists and available stock < ordered quantity	Create a Pending Restock Request for ordered quantity minus available stock.	Existing stock remains unchanged.
No usable product or nonpositive or missing quantity	Skip the record in the handler.	No stock update by this method.

Table 16: Order Conditions and Inventory Results

The handler collects product IDs, queries the products once, and uses a map for changes. If several orders reference one product in the same call, the shared in-memory product value is updated as the loop proceeds and the product ID is included only once in the update collection. This avoids the duplicate-ID DML problem in the original list-based sample.

Creation procedure

1. Open Developer Console from the Salesforce gear menu.
2. Choose File, New, Apex Class. Enter InventoryManager and create the class.
3. Replace the editor contents with the complete InventoryManager listing in the relevant source listing in this chapter and save it.
4. Choose File, New, Apex Trigger. Enter UpdateStockAfterOrder, select Re_Plastic_Innovations_Order__c and submit.
5. Paste the order trigger listing in the relevant source listing in this chapter and save. Confirm the Problems panel has no compilation error.
6. If the trigger reports that processOrderStock does not exist, first check that InventoryManager was saved successfully and that its method is public static with the matching custom Order list parameter.

Only after insert is implemented for orders. Updating an existing order quantity, deleting an order or cancelling an order does not reconcile stock in this version.

The complete InventoryManager listing below uses the actual Order.Product__c lookup. Code listings are documentation copies; match the saved org version to the final test evidence. Line numbers are displayed for review and must not be included when copying source into Developer Console.

InventoryManager

```
1 public class InventoryManager {
2
3     // Reduce stock after an order, or request the missing quantity.
4     public static void processOrderStock(
5         List<Re_Plastic_Innovations_Order__c> orders
6     ) {
7         Set<Id> productIds = new Set<Id>();
8
9         for (Re_Plastic_Innovations_Order__c orderRecord : orders) {
10             if (orderRecord.Product__c != null &&
11                 orderRecord.Quantity__c != null &&
12                 orderRecord.Quantity__c > 0) {
13                 productIds.add(orderRecord.Product__c);
14             }
15         }
16
17         if (productIds.isEmpty()) return;
18
19         Map<Id, Re_Plastic_Innovations_Recycled_Product__c> products =
20             new Map<Id, Re_Plastic_Innovations_Recycled_Product__c>{
```

```

21     [SELECT Id, Stock_Level_c
22     FROM Re_Plastic_Innovations_Recycled_Product_c
23     WHERE Id IN :productIds]
24 );
25
26 Map<Id, Re_Plastic_Innovations_Recycled_Product_c> changedProducts =
27     new Map<Id, Re_Plastic_Innovations_Recycled_Product_c>();
28
29 List<Re_Plastic_Innovations_Restock_Request_c> requests =
30     new List<Re_Plastic_Innovations_Restock_Request_c>();
31
32 for (Re_Plastic_Innovations_Order_c orderRecord : orders) {
33     Re_Plastic_Innovations_Recycled_Product_c product =
34         products.get(orderRecord.Product_c);
35
36     if (product == null ||
37         orderRecord.Quantity_c == null ||
38         orderRecord.Quantity_c <= 0) {
39         continue;
40     }
41
42     Decimal available = product.Stock_Level_c == null
43         ? 0 : product.Stock_Level_c;
44
45     if (available >= orderRecord.Quantity_c) {
46         product.Stock_Level_c =
47             available - orderRecord.Quantity_c;
48         changedProducts.put(product.Id, product);
49     } else {
50         requests.add(
51             new Re_Plastic_Innovations_Restock_Request_c(
52                 Product_c = product.Id,
53                 Requested_Quantity_c =
54                     orderRecord.Quantity_c - available,
55                 Status_c = 'Pending'
56             )
57         );
58     }
59 }
60
61 if (!changedProducts.isEmpty()) {
62     update changedProducts.values();
63 }
64
65 if (!requests.isEmpty()) {
66     insert requests;
67 }
68 }
69
70 // Add requested stock when an approved restock is passed in.
71 public static void processRestockApproval(
72     List<Re_Plastic_Innovations_Restock_Request_c> restocks
73 ) {
74     Set<Id> productIds = new Set<Id>();
75
76     for (Re_Plastic_Innovations_Restock_Request_c request : restocks) {
77         if (request.Status_c == 'Approved' &&
78             request.Product_c != null &&
79             request.Requested_Quantity_c != null &&
80             request.Requested_Quantity_c > 0) {
81             productIds.add(request.Product_c);
82         }
83     }
84
85     if (productIds.isEmpty()) return;
86
87     Map<Id, Re_Plastic_Innovations_Recycled_Product_c> products =
88         new Map<Id, Re_Plastic_Innovations_Recycled_Product_c>{
89             [SELECT Id, Stock_Level_c
90             FROM Re_Plastic_Innovations_Recycled_Product_c
91             WHERE Id IN :productIds]

```

```

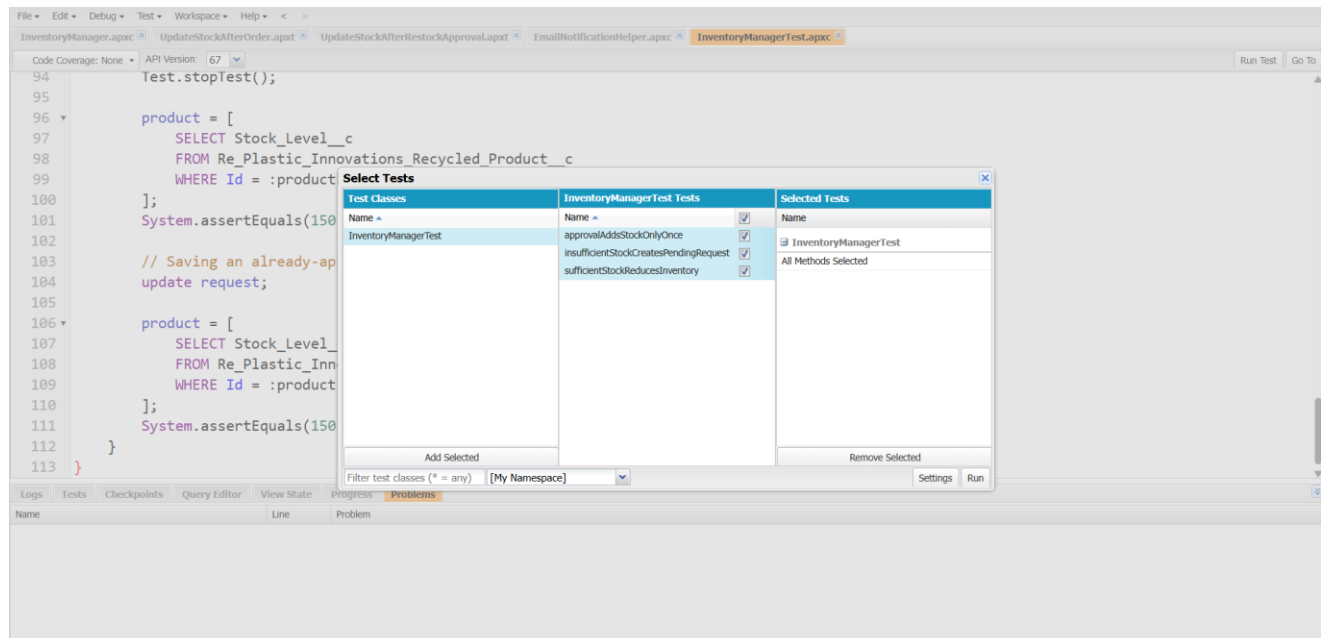
92     );
93
94     for (Re_Plastic_Innovations_Restock_Request__c request : restocks) {
95         if (request.Status__c != 'Approved' ||
96             request.Requested_Quantity__c == null ||
97             request.Requested_Quantity__c <= 0) {
98             continue;
99         }
100
101         Re_Plastic_Innovations_Recycled_Product__c product =
102             products.get(request.Product__c);
103
104         if (product == null) continue;
105
106         Decimal available = product.Stock_Level__c == null
107             ? 0 : product.Stock_Level__c;
108
109         product.Stock_Level__c =
110             available + request.Requested_Quantity__c;
111     }
112
113     update products.values();
114 }
115 }

```

4.4 Inventory Validation - UI Testing

Quantity validation is performed before a record is saved. On the custom Order object, an attempt to save Quantity 0 should display “Quantity must be greater than zero.” Change the value to a positive quantity, such as 2, and save with any other required fields populated.

An insufficient-stock order follows a different path: the current Apex handler allows the order insert and creates a Pending Restock Request for the missing units. It does not block the order as out of stock. With stock 50 and order quantity 150, the product remains at 50 and the requested quantity is 100.



4.5 Stock Reduction after a Valid Order

Use a separate test product with Stock Level 50. Create a new Order for 20 units through the Product lookup and save. The after-insert trigger calls `InventoryManager`, the available quantity is sufficient, and Stock Level becomes 30. No restock request should be created for this scenario.

Capture the starting product stock, the saved order quantity and the product after the transaction. Product identifiers should remain visible so the before-and-after values can be traced to the same record.

4.6 Apex Trigger Definitions

The order trigger executes after insert. The restock trigger executes after update and selects only a transition from a status other than Approved into Approved. The handlers process the related stock records; the approval trigger also invokes the email helper.

UpdateStockAfterOrder

```
1 trigger UpdateStockAfterOrder
2   on Re_Plastic_Innovations_Order__c (after insert) {
3
4     InventoryManager.processOrderStock(Triiger.new);
5   }
```

UpdateStockAfterRestockApproval

```
1 trigger UpdateStockAfterRestockApproval
2   on Re_Plastic_Innovations_Restock_Request__c (after update) {
3
4     List<Re_Plastic_Innovations_Restock_Request__c> newlyApproved =
5       new List<Re_Plastic_Innovations_Restock_Request__c>();
6
7     for (Re_Plastic_Innovations_Restock_Request__c request : Trigger.new) {
8       if (request.Status__c == 'Approved' &&
9         Trigger.oldMap.get(request.Id).Status__c != 'Approved') {
10        newlyApproved.add(request);
11      }
12    }
13
14    if (!newlyApproved.isEmpty()) {
15      InventoryManager.processRestockApproval(newlyApproved);
16      EmailNotificationHelper.sendRestockNotification(newlyApproved);
17    }
18  }
```

Save InventoryManager and EmailNotificationHelper successfully before saving a trigger that calls them. The reported “method does not exist or incorrect signature” error was resolved by using the saved public static processOrderStock method with the matching custom Order list parameter.

4.7 Restock Approval and Email Notification

UpdateStockAfterRestockApproval runs after a Restock Request is updated. It selects requests whose new Status is Approved and whose previous Status was not Approved. It then calls the stock-increase method and the email helper.

1. Create the EmailNotificationHelper Apex class and save its listing from the relevant source listing in this chapter.
2. Create an Apex trigger named UpdateStockAfterRestockApproval on Re_Plastic_Innovations_Restock_Request__c.
3. Use the after update trigger listing in the relevant source listing in this chapter. It must compare the old and new status before collecting requests.
4. Save the trigger after its referenced classes compile successfully.
5. Open a Pending restock request and change Status to Approved to exercise the approval transition.

Component	Implemented behavior
Stock update	Add Requested_Quantity__c to the related product Stock_Level__c; blank stock is treated as zero.
Recipients	Active Salesforce users whose role Name is Warehouse Supervisor and whose Email is populated.
Email subject	Restock Request Approved - Stock Updated
Email body	Request name, product ID and quantity added for the approved requests in the call.
Ordinary repeated save	Saving the request again while Status stays Approved does not pass it to the handler again.

Table 17: Restock Approval and Notification Behavior

The email is directed to warehouse-role users, not automatically to the customer or order owner. Actual inbox delivery was not confirmed in the supplied progress; the final email screenshot remains to be captured. Test coverage of email code is not evidence that a real email was delivered.

Important limit: moving an approved request to another status and later back to Approved can add the quantity again. The current trigger prevents a duplicate increment on an unchanged Approved save, but it does not implement permanent once-only processing.

EmailNotificationHelper

```

1  public class EmailNotificationHelper {
2
3      public static void sendRestockNotification(
4          List<Re_Plastic_Innovations_Restock_Request__c> approvedRequests
5      ) {
6          if (approvedRequests.isEmpty()) return;
7
8          List<String> recipients = new List<String>();
9
10         for (User warehouseUser : [
11             SELECT Email
12             FROM User
13             WHERE IsActive = true
14             AND UserRole.Name = 'Warehouse Supervisor'
15             AND Email != null
16         ]) {
17             recipients.add(warehouseUser.Email);
18         }
19
20         if (recipients.isEmpty()) return;
21
22         String body =
23             'Stock was updated for these approved restock requests:\n';
24
25         for (Re_Plastic_Innovations_Restock_Request__c request
26             : approvedRequests) {
27             body += 'Request: ' + request.Name
28                 + ', Product ID: ' + request.Product__c
29                 + ', Quantity added: ' + request.Requested_Quantity__c
30                 + '\n';
31         }
32
33         Messaging.SingleEmailMessage message =
34             new Messaging.SingleEmailMessage();
35         message.setToAddresses(recipients);
36         message.setSubject('Restock Request Approved - Stock Updated');
37         message.setPlainTextBody(body);
38
39         Messaging.sendEmail(
40             new List<Messaging.SingleEmailMessage>{ message }
41         );
42     }
43 }
```

Restock Approval Verification

1. Create a Recycled Product named Plastic Order Test Product. Set Stock Level to 50, Threshold to 100 and Price to 500. Save and capture the starting values.
2. Create a new custom Order linked through Product to this record. Set Quantity to 150, populate any other required fields, and save.
3. Open Restock Requests and find the request for that product. Verify Requested Quantity 100 and Status Pending. Reopen the product and verify stock is still 50.
4. Edit the request and change Status from Pending to Approved. Save once, refresh the product, and verify Stock Level 150.
5. Save the request again without changing Approved. Verify stock remains 150. Check for the warehouse notification separately using Section 4.2 or Section 4.7.

This scenario demonstrates shortage and replenishment, not automatic fulfillment of the original insufficient-stock order. The available 50 units are not reserved or deducted when the insufficient order is inserted.

Approval-email delivery still requires inbox evidence. The helper selects active users in the Warehouse Supervisor role. Verify the recipient and actual message after an approval; automated test coverage does not demonstrate live email delivery.

4.8 Formula Field - Stock Low On Product

1. Open Object Manager, select Re Plastic Innovations Recycled Product, then Fields & Relationships and New.
2. Select Formula. Enter Stock Low On Product as Field Label and choose Text as the return type.
3. Click Next and open Advanced Formula. Enter the expression below.
4. Click Check Syntax. Continue through field visibility and page layout selection, then save.
5. Open a product record and verify that the result changes when Stock Level is edited.

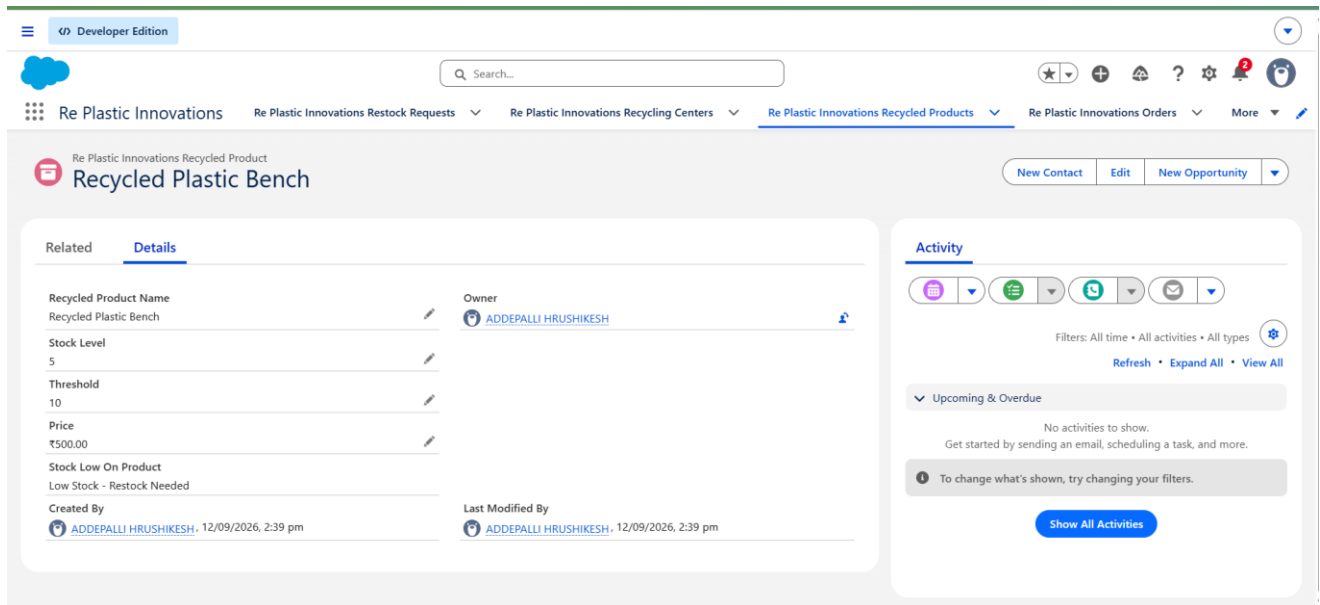
```
IF(  
  Stock_Level_c < Threshold_c,  
  "Low Stock - Restock Needed",  
  "Sufficient Stock"  
)
```

Field API name: Stock_Low_On_Product_c. The comparison is strictly less than; stock equal to the threshold returns Sufficient Stock.

Test values	Expected formula output	Evidence status
Stock 5; Threshold 10	Low Stock - Restock Needed	Walkthrough test; capture result.
Stock 15; Threshold 10	Sufficient Stock	Walkthrough test; capture result.
Stock 10; Threshold 10	Sufficient Stock	Boundary expectation; not separately confirmed.

Table 18: Low Stock Formula Test Values and Expected Output

Example product used in the procedure: Recycled Plastic Bench, with Price 500. The formula is display logic only; it does not change stock, create a Task or insert a restock request by itself.



4.9 Validation Rules

Order Quantity Validation

1. In Object Manager, open Re Plastic Innovations Order.
2. Select Validation Rules and click New.
3. Enter Check_Quantity_Not_Zero as Rule Name and enable Active.
4. Enter the Error Condition Formula shown below and check its syntax.
5. Set the error message to Quantity must be greater than zero.
6. Select Field as the error location and choose Quantity. Save.

Quantity_c <= 0

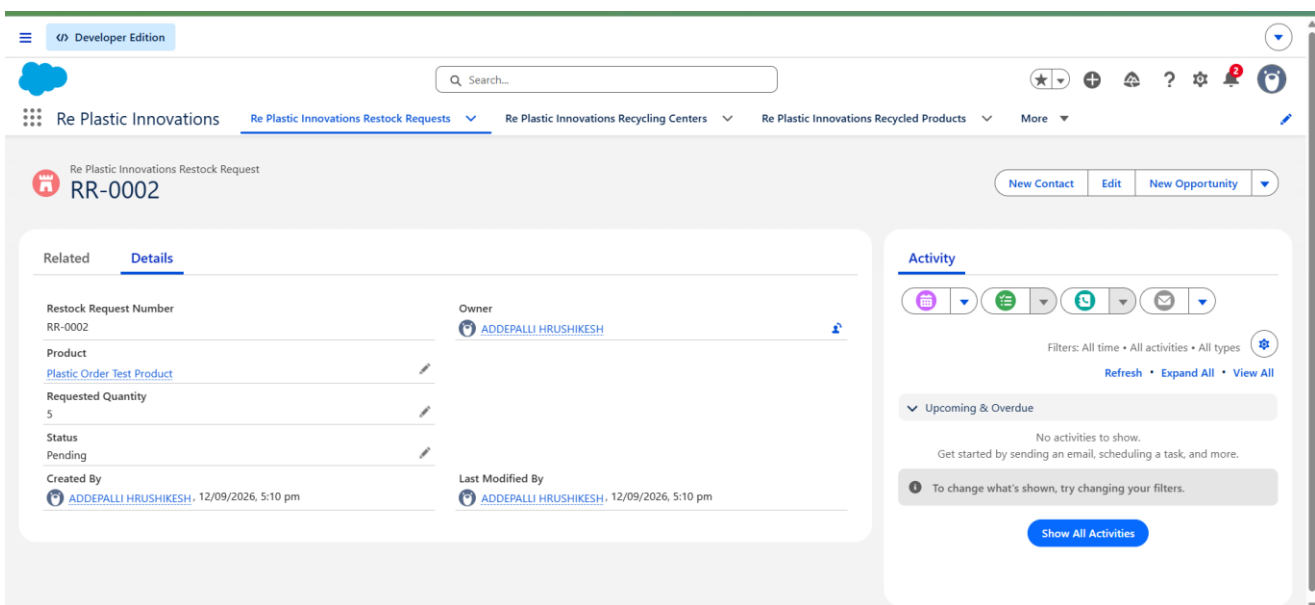
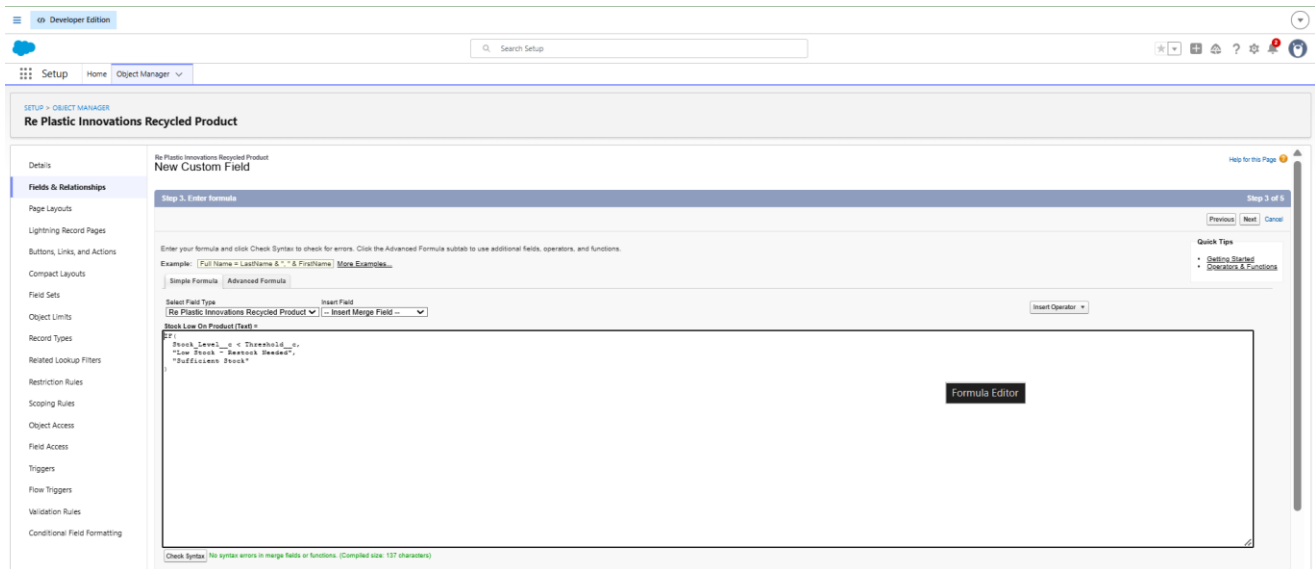
Rule setting	Value
Rule Name	Check_Quantity_Not_Zero
Active	True
Error Message	Quantity must be greater than zero.
Error Location	Field: Quantity

Table 19: Order Quantity Validation Rule

Test procedure

Create an Order using the project app. Enter Quantity 0 and try to save: the error should appear beside Quantity. Change Quantity to a positive value, such as 2, and save with any other required fields populated. "Change Quality" in the supplied instruction refers to Quantity.

This rule rejects nonpositive quantities; it is not a replacement for all order completeness rules. The project did not add a separate product-required or delivery-date validation rule here.



Plastic Waste Collection Date Validation

1. In Object Manager, open Re Plastic Innovations Plastic Waste.
2. Select Validation Rules, click New, and enter Future_Date_Collection.
3. Enable Active, enter the error condition formula below and check syntax.
4. Set the error message to Collection Date cannot be in the future.
5. Set the error location to Field and choose Collection Date. Save.

Collection_Date_c > TODAY()

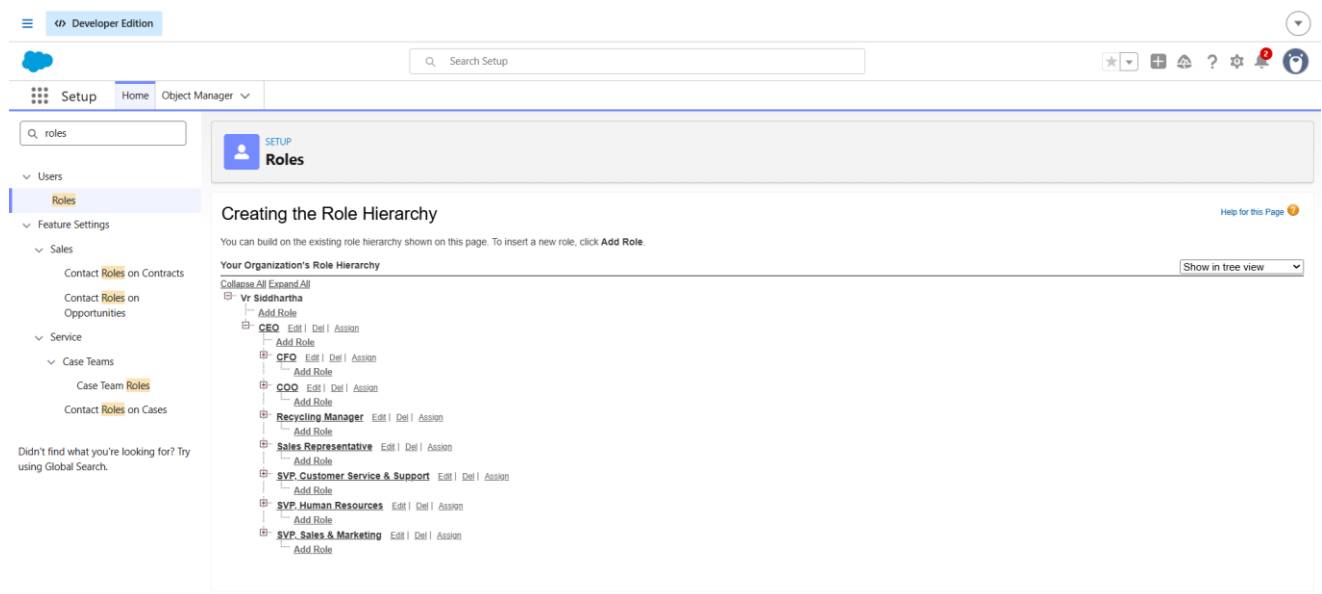
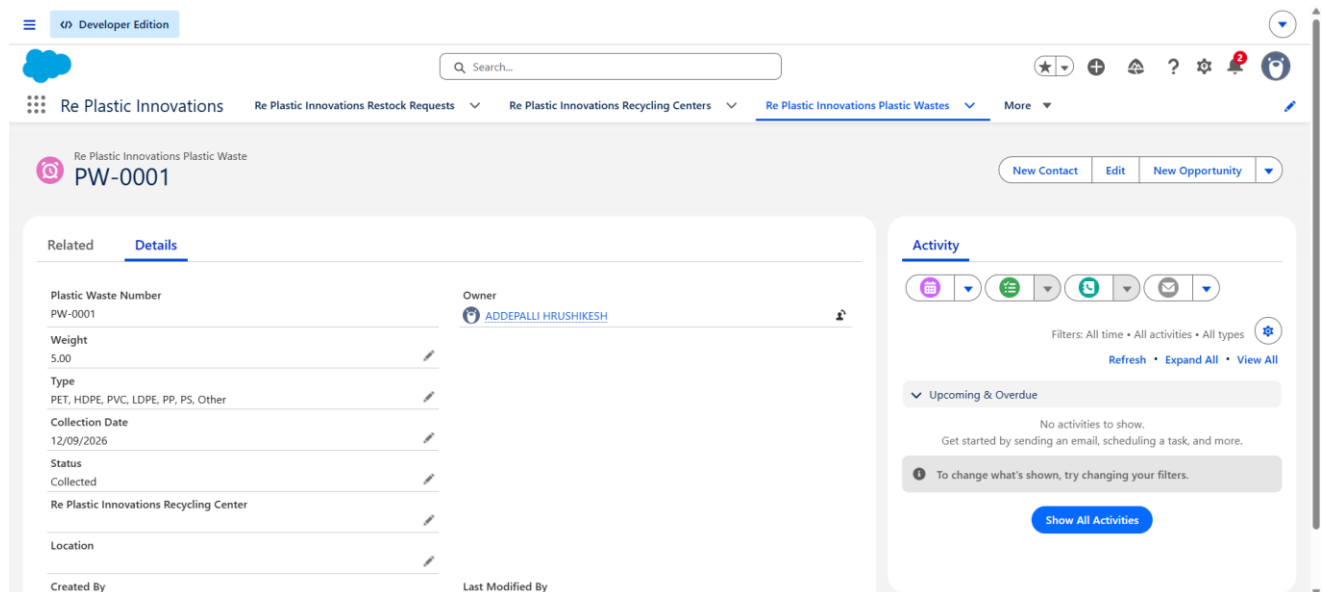
Rule setting	Value
Rule Name	Future_Date_Collection
Active	True
Error Message	Collection Date cannot be in the future.
Error Location	Field: Collection_Date_c

Table 20: Collection Date Validation Rule

Test procedure

Create a Plastic Waste record with Weight 5, Type PET and Status Collected. Set Collection Date to tomorrow and attempt to save. The configured error should block the save. Change the date to today and save. Populate any other fields required by the current layout.

TODAY() is evaluated at runtime; do not hard-code the date from an old screenshot. The rule blocks future dates but does not make a blank date mandatory. The positive and negative screenshots should refer to the same test scenario.



4.10 Roles, Profiles and User Creation

Role Hierarchy

Roles describe reporting relationships and can affect record visibility through the hierarchy. They are separate from profiles, which control object and field capabilities.

Role label	This role reports to	Project responsibility
Recycling Manager	CEO	Waste collection oversight.
Sales Representative	CEO	Customer and recycled product orders.
Warehouse Supervisor	Sales Representative	Restock and warehouse follow-up.

Table 21: Project Role Hierarchy

1. In Setup, search Roles. Open the role hierarchy.
2. Use Add Role below CEO. Enter Recycling Manager as the Label and confirm This Role Reports To is CEO. Save.
3. Add Sales Representative below CEO and save.
4. Add Warehouse Supervisor below Sales Representative and save.
5. If a role is reported as already existing but is not visible in the current tree, change or refresh the hierarchy view and inspect the existing role. Do not create a duplicate role with a slightly different name.

Interpretation

The role names follow the supplied instructions even though some sample user first names refer to production or inspection jobs. Keep role labels exact because the email helper uses the Warehouse Supervisor role name to find recipients.

Custom Profile Creation

Three custom profiles were created by cloning Standard Platform User. The Salesforce Platform user license determines which capabilities are available to these profiles.

Profile name	Existing profile to clone	User license
Platform 1	Standard Platform User	Salesforce Platform
Platform 2	Standard Platform User	Salesforce Platform
Platform 3	Standard Platform User	Salesforce Platform

Table 22: Custom Profiles and Salesforce Platform Licenses

1. From Setup, search Profiles and open the profile list.
2. Click New Profile, or use Clone on Standard Platform User if the interface presents that route.
3. Select Standard Platform User as Existing Profile and enter Platform 1 as the Profile Name. Save.
4. Repeat the same process for Platform 2 and Platform 3.

- Open each custom profile for the object permission changes in the next subsection. Do not modify the standard source profile.
- Keep unrelated inherited settings unchanged unless the project specifically requires a change.

The Standard Object Permissions section contains Accounts. The project custom objects appear in Custom Object Permissions, farther down the classic profile edit page, or within Object Settings in an enhanced profile interface.

User Creation

User details	Role	Profile and license
First Name: John Production Engineer Last Name: Sandbox 1.	Recycling Manager	Platform 1 Salesforce Platform
First Name: Quality Inspector Last Name: Mike	Sales Representative	Platform 2 Salesforce Platform
First Name: Plant Manager Last Name: Albert	Warehouse Supervisor	Platform 3 Salesforce Platform

Table 23: Sample Users with Role and Profile Assignments

- In Setup, search Users and open Users. Click New User.
- Enter the first and last names from the table. Review the automatically populated Alias.
- Enter an accessible personal email address for the learning account. Use a globally unique username in email format; it does not have to be a working mailbox.
- Select the required Role, Salesforce Platform User License and matching Platform profile. Confirm a license is available.
- Save the user and send the initial password email as part of user creation where that option is shown.
- Open the Salesforce account activation or password setup email, follow the link, set the password and complete activation.
- Repeat for the other two users, then verify the role and profile assignments in the user list.

The example username in the original guide was illustrative. Use the actual unique username saved in the org, not a shared example. Do not place passwords, activation links or full personal email addresses in the submission.

The user creation activities were followed in the walkthrough. A final redacted Users screenshot should be attached to evidence the saved assignments and active status; this document does not reproduce private login information.

The screenshot shows the Salesforce Setup interface with the 'Users' section selected. The 'New User' form is displayed, showing the following details:

- General Information:**
 - First Name: John Production Engineer
 - Last Name: Sandbox 1
 - Alias: jsand
 - Email: hrushikeshadd2005@gmail.com
 - Username: hrushikeshadd2005@gmail.com
 - Nickname: User178920084071581373
 - Title: (empty)
 - Company: (empty)
 - Department: (empty)
 - Division: (empty)
- Role:**
 - Role: Recycling Manager
 - User License: Salesforce Platform
 - Profile: Platform 1
 - Active: ☒
 - Marketing User: ☐
 - Offline User: ☐
 - Knowledge User: ☐
 - Flow User: ☐
 - Service Cloud User: ☐
 - Site.com Contributor User: ☐
 - Site.com Publisher User: ☐

4.11 Object-Level and Record-Level Access

Profile Object Permissions

The table records the requested project permissions and the completion boundary. R means Read, C means Create and E means Edit. A dash means no additional permission was specified in that activity; it does not prove the cloned profile has no inherited access.

Object	Platform 1	Platform 2	Platform 3 target
Plastic Waste	R C	R	R C E
Recycling Center	—	—	R C E deferred
Recycled Product	—	R	R C E deferred
Order custom object	—	R C	R C E deferred
Restock Request	R	—	R C E deferred
Account standard object	—	R C	Not specified

Table 24: Project Object Permission Matrix

1. Open Platform 1 and click Edit. In Custom Object Permissions, enable Read and Create for Plastic Waste, and Read only for Restock Request. Save.
2. Open Platform 2. Enable Read and Create for the custom Order object and for standard Accounts. Enable Read only for Plastic Waste and Recycled Product. Save.
3. For the Accounts row, remove inherited Edit and Delete when applying the requested Read/Create scope. Leave View All and Modify All unselected for these project access grants.
4. Platform 3 was configured for Plastic Waste Read/Create/Edit. The learner explicitly deferred applying the same permissions to the other four custom objects. Record the target, but do not mark those four grants complete.
5. Do not grant Delete, View All Records, Modify All Records or View All Fields merely to resolve a visibility issue. First check the intended object, field and record access.

Deferred item: complete the remaining Platform 3 grants only when the learner chooses to resume them. Field-level security, app assignment and record-level access still require a real user access check; the Apex test results do not validate those settings.

The screenshot shows the Salesforce Setup interface for the 'Profiles' section. The page title is 'Platform 1'. It lists various permissions like 'Login IP Ranges', 'Enabled Apex Class Access', 'Enabled Visualforce Page Access', 'Enabled External Data Source Access', 'Enabled Named Credential Access', 'Enabled External Credential Principal Access', 'Enabled Custom Metadata Type Access', 'Enabled Custom Setting Definitions Access', 'Enabled Flow Access', 'Enabled Service Presence Status Access', and 'Enabled Custom Permissions'. Below the permissions, there's a 'Profile Detail' section with fields for Name, User License, Description, Created By, and Modified By. At the bottom, there's a 'Page Layouts' section showing 'Standard Object Layouts' for Global, Email Application, and Home Page Layout.

Organization-Wide Defaults

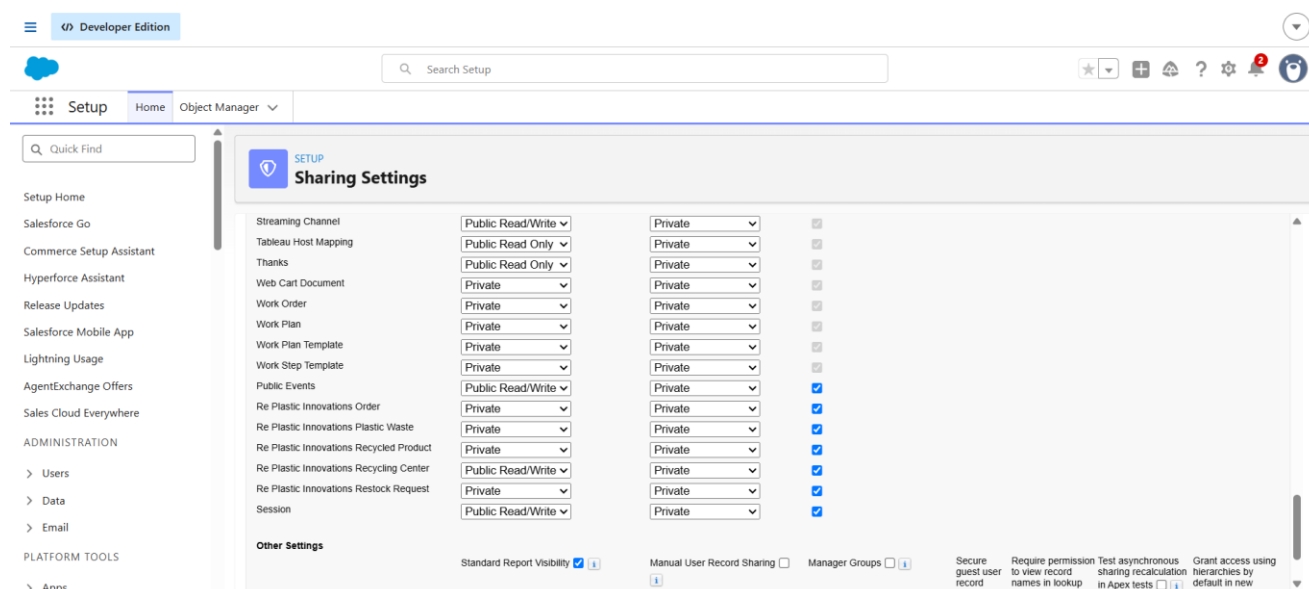
Organization-wide defaults establish baseline record visibility. The walkthrough used Private for the three objects involved in selective sharing. The other objects were not assigned a new baseline in the supplied sharing activity.

Custom object	Internal access baseline in procedure	Reason
Plastic Waste	Private	Allow selected waste records to be shared to Recycling Manager.
Recycled Product	Private	Allow selected product records to be shared to Sales Representative.
Restock Request	Private	Allow selected requests to be shared to Warehouse Supervisor.
Recycling Center	Unchanged; verify saved value	No selective sharing change specified.
Order	Unchanged; verify saved value	No selective sharing change specified.

Table 25: Organization-Wide Default Access Settings

1. In Setup, search Sharing Settings and open it.
2. Click Edit in Organization-Wide Defaults. Find the correct project custom object rows.
3. Set the internal default access to Private for Plastic Waste, Recycled Product and Restock Request as used for the sharing exercise. Save.
4. Allow Salesforce to complete sharing recalculation. If a yellow update banner appears and New is disabled for an object, wait for that operation and refresh the page.
5. After the update finishes, create the owner-based rules in the next subsection. Do not repeatedly change the defaults to force the button to become available.

Sharing rules add access; they do not restrict existing access. “Only the Recycling Manager” in the learning description is not an absolute security guarantee: record owners, administrators, hierarchy access and other grants can still provide visibility. The hierarchy setting was not disabled in this walkthrough. [R1]



Owner-Based Sharing Rules

Use the specific Roles option for both the record owner group and the recipient group. Do not silently broaden either selection to Roles and Subordinates.

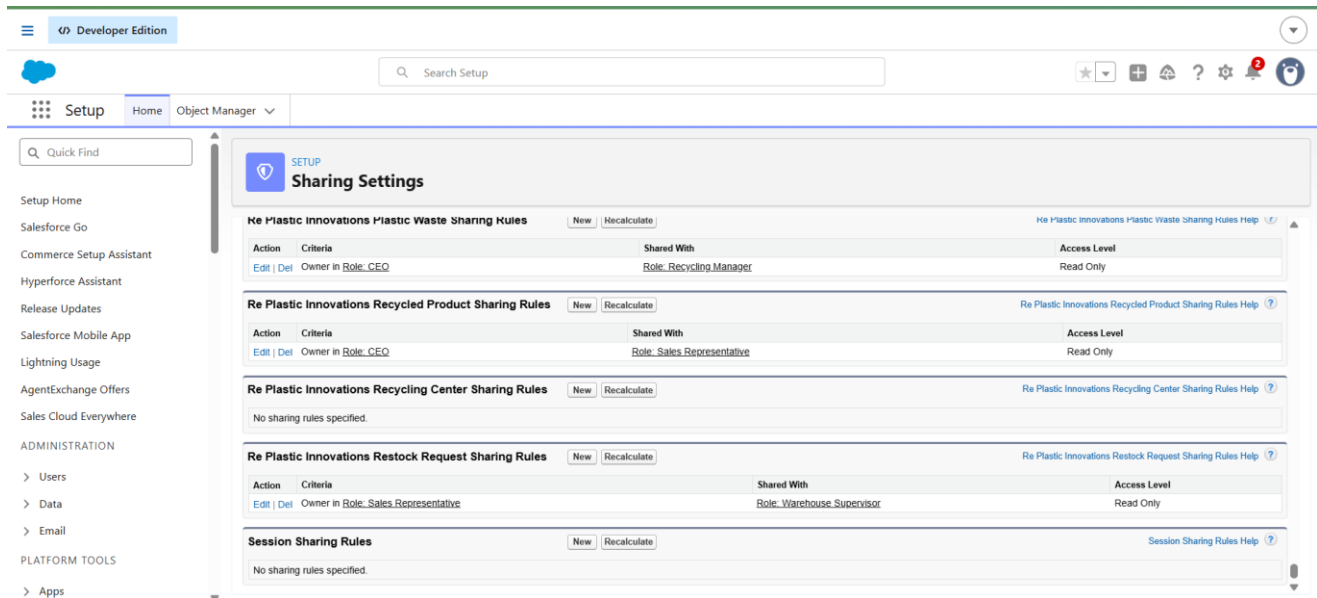
Object and rule label	Owned by role	Share with role	Access
Plastic Waste Recycling Manager Access	CEO	Recycling Manager	Read Only
Recycled Product Sales Representative Product Access	CEO	Sales Representative	Read Only
Restock Request Warehouse Supervisor Restock Access	Sales Representative	Warehouse Supervisor	Read Only

Table 26: Owner-Based Sharing Rules and Access Levels

1. In Sharing Settings, locate the sharing rules section for the relevant custom object and click New.
2. Enter the rule Label and accept or enter its API Rule Name. For the first rule, use Recycling_Manager_Access.
3. Select Based on record owner.
4. Under Select which records to be shared, select Roles and then the owner role from the table.
5. Under Select the users to share with, select Roles and then the recipient role from the table.
6. Select Read Only and save. Confirm any recalculation prompt.
7. Repeat for the product and restock objects, using Sales_Representative_Product_Access and Warehouse_Supervisor_Restock_Access as the rule names.

The reference to “Steel Production” in the supplied instructions was a copied heading unrelated to this project. The correct first target is Re Plastic Innovations Plastic Waste. No Steel Production object was needed or created for this requirement.

The rule only shares records owned by users in its source role. It does not automatically share every record in the object. Recipients still need object Read permission, so the deferred Platform 3 settings remain relevant.



5 Phase 3: UI/UX Development & Customization

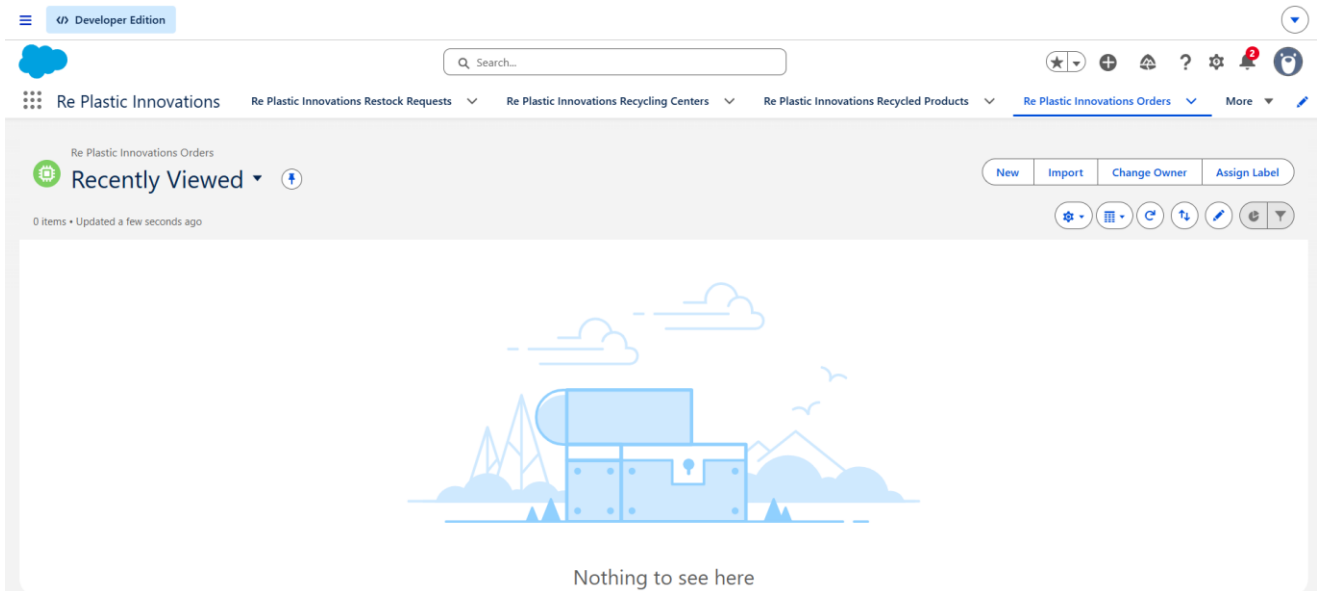
5.1 Lightning App & Tab Setup

1. In Setup, search App Manager, open it and click New Lightning App.
2. Enter the App Name, accept the automatically generated Developer Name, and add the description shown below.
3. Leave the image optional and the primary color at its default. Click Next.
4. Keep the project defaults on App Options. Continue through Utility Items without adding a utility.
5. On Navigation Items, search for the five custom object tabs and move each to Selected Items.
6. On User Profiles, select System Administrator, move it to the selected list, and choose Save & Finish.
7. Open the App Launcher, search Re Plastic Innovations and verify that the app opens with the selected navigation items.

Setting	Value
App Name	Re Plastic Innovations
Developer Name	Re_Plastic_Innovations, automatically generated
Description	Manage plastic waste collection, recycling centers, recycled products, customer orders and restock requests to support a circular economy.
Navigation items	Restock Requests, Recycling Centers, Recycled Products, Plastic Wastes and Orders; select the project custom tabs.
Assigned profile in app wizard	System Administrator

Table 27: Re Plastic Innovations Lightning App Configuration

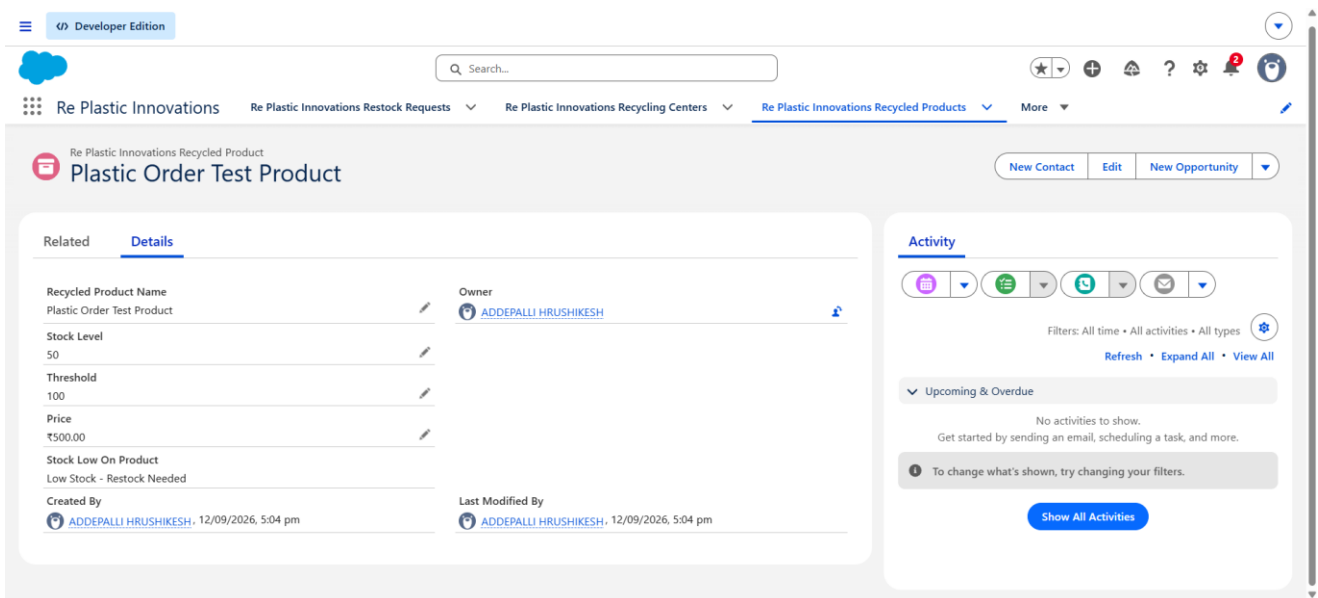
App visibility for Platform 1, Platform 2 and Platform 3 was not separately verified; profile object permissions alone do not demonstrate that those users can launch this app.



5.2 Record Pages Used During Testing

Standard Lightning record pages and page layouts expose the project fields. Plastic Waste pages display collection information and the assigned center; Recycling Center pages display coordinates and capacity; Product pages display stock, threshold, price and the low-stock formula; Order pages display Account, Product, Quantity and Delivery Date; and Restock Request pages display Product, Requested Quantity and Status.

No separate Dynamic Forms rules or custom Lightning components were implemented. Allow Activities supports the relationship between the scheduled Task and the custom product. Field-level security and app assignment for the cloned Platform profiles still require verification through the corresponding users.



6 Phase 4: Data Handling, Testing & Quality Assurance

6.1 Test Data Preparation

Functional test records were entered through the Salesforce Lightning application. Apex tests create their own product and order data. No bulk data import was performed. Separate product records should be used for independent scenarios so a previous order does not change the expected starting inventory.

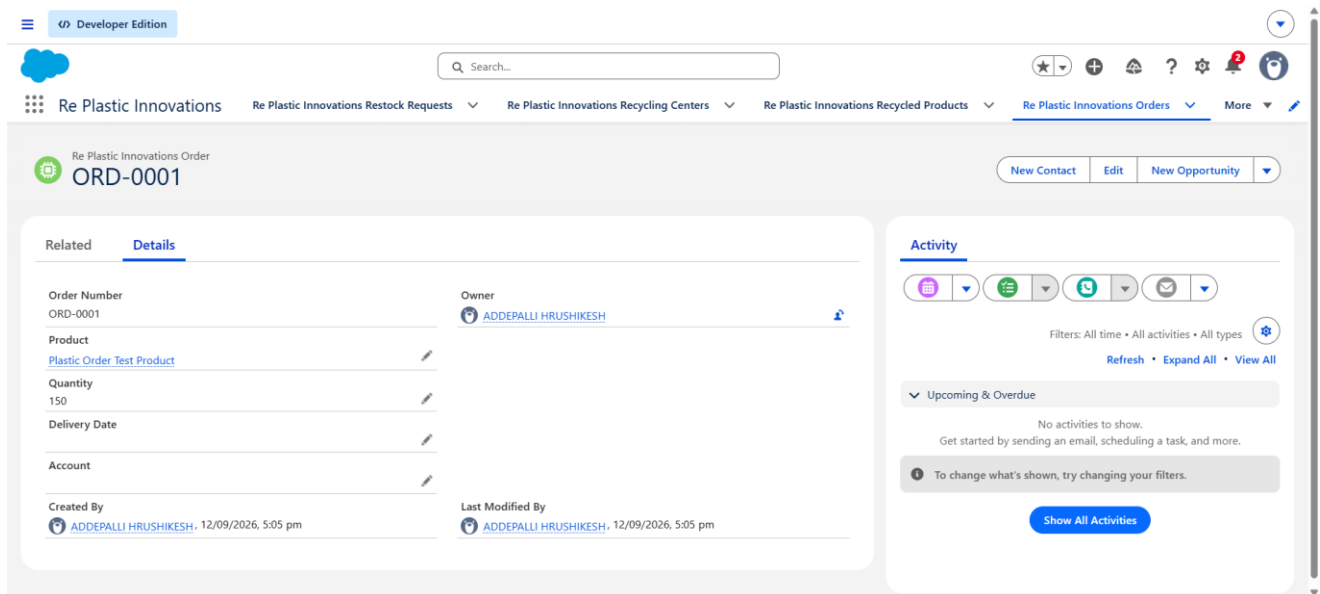
The product used by InventoryManagerTest is named Apex Test Product, with stock 50, threshold 10 and price 500. The product Name is supplied because it is a Text record name. Order and Restock Request use auto-number names and therefore do not assign Name manually in the tests.

6.2 Functional Testing Matrix

Use separate test products for independent scenarios so earlier orders do not change the starting stock. Populate required fields and lookups. The table separates automated assertions from manual evidence still to be attached.

Test	Input or action	Expected result	Evidence
T01 Low-stock formula	Stock 5; threshold 10	Low Stock - Restock Needed	Manual capture required
T02 Sufficient-stock formula	Stock 15; threshold 10	Sufficient Stock	Manual capture required
T03 Order validation	Quantity 0, then 2	Zero blocked; positive value accepted	Manual capture required
T04 Waste date validation	Tomorrow, then today	Future blocked; today accepted	Manual capture required
T05 Sufficient order	Initial stock 50; order 20	Stock 30; no restock request	Apex assertion
T06 Insufficient order	Initial stock 50; order 150	Stock stays 50; Pending request 100	Apex assertion
T07 Approve restock	Approve the request for 100	Stock becomes 150	Apex assertion
T08 Save approved again	Save without changing Approved	Stock remains 150	Apex assertion
T09 Mixed valid and skipped input	Direct handler calls: order 5 and 0; approved restock 10 and pending 20	Starting stock 50 becomes 55	Guard regression case
T10 Daily Flow	Run schedule for a low-stock product	High priority related Task	Runtime result unconfirmed
T11 Approval email	Approve a request with an active warehouse-role user	Email reaches warehouse inbox	Delivery unconfirmed
T12 User access	Sign in as each sample user	Access matches configured object, field and record grants	Access check not completed

Table 28: System Testing and Quality Assurance Matrix



6.3 Apex Test Coverage Scope

Test class procedure

1. In Developer Console, choose File, New, Apex Class. Name it InventoryManagerTest.
2. Use the complete test class listing in the relevant source listing in this chapter as the documented test reference. The three core cases insert or update records so the triggers run naturally.
3. Save the class. Choose Run Test from the class view or Test, New Run, and select InventoryManagerTest.
4. Wait for the run to finish. In Tests, open the latest run and check that each selected method passed.
5. Review Overall Code Coverage and the InventoryManager entry. Open the class with the applicable coverage view to inspect uncovered lines.
6. Capture the final passing run and the coverage table together. Do not use the older 96 percent screenshot as the final evidence.

Coverage issue and correction

The supplied screenshot showed InventoryManager at 62 of 64 lines, displayed as 96 percent, while EmailNotificationHelper showed 100 percent. The learner identified highlighted continue statements. The guard regression case uses valid and invalid records in the same handler call so the method does not exit before the skip branch can be reached.

Invalid test-only records are passed directly to the handler without insertion. This avoids the Order validation rule blocking Quantity 0 before the Apex skip logic can be tested. Do not call processOrderStock a second time for a valid order that already fired the after-insert trigger; that would process inventory twice.

Reported final result

The final project completion confirmation reported 100 percent coverage. Attach the latest passing test run and the saved class coverage as evidence of that result.

Coverage measures executed code, not proof of every business scenario. The reported result does not establish live email delivery, scheduled Flow execution, permissions correctness, concurrency safety or production deployment readiness.

InventoryManagerTest

The test reference below retains the core sufficient-stock, shortage and approval scenarios and the mixed-input guard case. The first three cases use DML to exercise the triggers; the guard case calls the handler directly with records that are intentionally not inserted. An existing order must not be processed again by a direct handler call after its trigger has already run.

```

1  @IsTest
2  private class InventoryManagerTest {
3
4      private static Re_Plastic_Innovations_Recycled_Product__c
5          createProduct(Decimal stock) {
6
7          Re_Plastic_Innovations_Recycled_Product__c product =
8              new Re_Plastic_Innovations_Recycled_Product__c(
9                  Name = 'Apex Test Product',
10                 Stock_Level__c = stock,
11                 Threshold__c = 10,
12                 Price__c = 500
13             );
14             insert product;
15             return product;
16         }
17
18     @IsTest
19     static void sufficientStockReducesInventory() {
20         Re_Plastic_Innovations_Recycled_Product__c product =
21             createProduct(50);
22
23         Test.startTest();
24         insert new Re_Plastic_Innovations_Order__c(
25             Product__c = product.Id,
26             Quantity__c = 20
27         );
28         Test.stopTest();
29
30         product = [
31             SELECT Stock_Level__c
32             FROM Re_Plastic_Innovations_Recycled_Product__c
33             WHERE Id = :product.Id
34         ];
35         System.assertEquals(30, product.Stock_Level__c);
36
37         Integer requestCount = [
38             SELECT COUNT()
39             FROM Re_Plastic_Innovations_Restock_Request__c
40             WHERE Product__c = :product.Id
41         ];
42         System.assertEquals(0, requestCount);
43     }
44
45     @IsTest
46     static void insufficientStockCreatesPendingRequest() {
47         Re_Plastic_Innovations_Recycled_Product__c product =
48             createProduct(50);
49
50         Test.startTest();
51         insert new Re_Plastic_Innovations_Order__c(
52             Product__c = product.Id,
53             Quantity__c = 150
54         );
55         Test.stopTest();
56
57         product = [
58             SELECT Stock_Level__c
59             FROM Re_Plastic_Innovations_Recycled_Product__c
60             WHERE Id = :product.Id
61         ];
62         System.assertEquals(50, product.Stock_Level__c);
63
64         List<Re_Plastic_Innovations_Restock_Request__c> requests = [
65             SELECT Requested_Quantity__c, Status__c
66             FROM Re_Plastic_Innovations_Restock_Request__c
67             WHERE Product__c = :product.Id
68         ];
69         System.assertEquals(1, requests.size());
70         System.assertEquals(100, requests[0].Requested_Quantity__c);
71         System.assertEquals('Pending', requests[0].Status__c);

```

```

72 }
73
74 @IsTest
75 static void approvalAddsStockOnlyOnce() {
76     Re_Plastic_Innovations_Recycled_Product__c product =
77         createProduct(50);
78
79     insert new Re_Plastic_Innovations_Order__c(
80         Product__c = product.Id,
81         Quantity__c = 150
82     );
83
84     Re_Plastic_Innovations_Restock_Request__c request = [
85         SELECT Id, Status__c
86         FROM Re_Plastic_Innovations_Restock_Request__c
87         WHERE Product__c = :product.Id
88         LIMIT 1
89     ];
90
91     Test.startTest();
92     request.Status__c = 'Approved';
93     update request;
94     Test.stopTest();
95
96     product = [
97         SELECT Stock_Level__c
98         FROM Re_Plastic_Innovations_Recycled_Product__c
99         WHERE Id = :product.Id
100     ];
101     System.assertEquals(150, product.Stock_Level__c);
102
103     // A save with unchanged Approved status must not add stock again.
104     update request;
105
106     product = [
107         SELECT Stock_Level__c
108         FROM Re_Plastic_Innovations_Recycled_Product__c
109         WHERE Id = :product.Id
110     ];
111     System.assertEquals(150, product.Stock_Level__c);
112 }
113
114 @IsTest
115 static void skipsInvalidOrdersAndUnapprovedRestocks() {
116     Re_Plastic_Innovations_Recycled_Product__c product =
117         createProduct(50);
118
119     Test.startTest();
120
121     // Direct calls avoid inserting intentionally invalid records.
122     InventoryManager.processOrderStock(
123         new List<Re_Plastic_Innovations_Order__c>{
124             new Re_Plastic_Innovations_Order__c(
125                 Product__c = product.Id,
126                 Quantity__c = 5
127             ),
128             new Re_Plastic_Innovations_Order__c(
129                 Product__c = product.Id,
130                 Quantity__c = 0
131             )
132         }
133     );
134
135     InventoryManager.processRestockApproval(
136         new List<Re_Plastic_Innovations_Restock_Request__c>{
137             new Re_Plastic_Innovations_Restock_Request__c(
138                 Product__c = product.Id,
139                 Requested_Quantity__c = 10,
140                 Status__c = 'Approved'
141             ),
142             new Re_Plastic_Innovations_Restock_Request__c(

```

```

143         Product__c = product.Id,
144         Requested_Quantity__c = 20,
145         Status__c = 'Pending'
146     )
147 }
148 );
149
150 Test.stopTest();
151
152 product = [
153     SELECT Stock_Level__c
154     FROM Re_Plastic_Innovations_Recycled_Product__c
155     WHERE Id = :product.Id
156 ];
157 System.assertEquals(55, product.Stock_Level__c);
158 }
159 }

```

The screenshot displays the Salesforce Developer Console interface. The top pane shows the Apex test class `InventoryManagerTest.apxc` with the following code:

```

141         new Re_Plastic_Innovations_Restock_Request__c(
142             Product__c = product.Id,
143             Requested_Quantity__c = 20,
144             Status__c = 'Pending'
145         )
146     }
147 );
148
149 Test.stopTest();
150
151 product = [
152     SELECT Stock_Level__c
153     FROM Re_Plastic_Innovations_Recycled_Product__c
154     WHERE Id = :product.Id
155 ];
156 System.assertEquals(55, product.Stock_Level__c);
157 }
158 }

```

The bottom pane shows the test results table:

Status	Test Run	Enqueued Time	Duration	Failures	Total	Overall Code Coverage
✓	TestRun @ 5:18:54 pm			0	3	100%
✓	TestRun @ 5:26:06 pm			0	3	100%
✓	TestRun @ 5:29:22 pm			0	4	100%

Below the test results table, the 'Overall Code Coverage' section provides a detailed breakdown:

Class	Percent	Lines
Overall	100%	
EmailNotificationHelper	100%	21/21
InventoryManager	100%	64/64
UpdateStockAfterOrder	100%	1/1
UpdateStockAfterRestockApproval	100%	9/9

7 Phase 5: Deployment, Documentation & Maintenance

7.1 Deployment Readiness

The project was configured and tested in a Salesforce Developer Edition org. A production deployment was not performed. A later move to another org should keep the dependent metadata together and verify target-org settings before operational use.

1. Move custom objects, fields, record-name settings, lookup relationships and picklist values.
2. Include the custom tabs, Lightning app and page layouts.
3. Include the required role, profile and sharing configuration; complete the explicitly deferred Platform 3 grants only when that activity is resumed.
4. Move the Flow, InventoryManager, EmailNotificationHelper, both triggers and the test class.

5. Activate the appropriate Flow version and verify its schedule and organization time zone in the destination org.
6. Rerun the automated tests and verify user access, stock transactions, a real scheduled Task and a real approval email.

Documentation Evidence

Insert screenshots in the numbered Figure spaces beside the corresponding configuration or test. Replace the blank area with an inline picture, keep the caption, and resize proportionally. Use final saved settings, readable API names and actual numerical results. Redact private usernames, email addresses and activation links. Correct the page numbers in Contents if inserting images changes pagination.

7.2 System Monitoring & Troubleshooting

Maintain the five objects and exact API names together with the automation. Before changing picklist values, role labels or lookup names, review the formula, Flow, Apex classes and tests that reference them. Preserve a copy of the saved org code before future changes.

For an inventory issue, inspect the order quantity, product lookup, stock value and related requests, then review Developer Console errors or debug logs. For a Flow issue, confirm the active version, the organization time zone and scheduled execution, the decision values, activities support and the Task field mapping. For an email issue, verify the active warehouse-role user and recipient address, org email settings and the actual inbox; do not change permissions indiscriminately.

No deployment to another org was carried out. A later deployment should include schema dependencies, application metadata, access configuration, Flow, Apex classes, triggers and tests, followed by target-org scheduling and end-to-end checks.

Implementation Corrections

The report uses the corrected project meaning rather than copying errors, duplicated activities or unrelated interface text from the source instructions.

Issue in instructions or setup	Correction or documented decision
Name appears in addition to the requested fields	Keep the standard Name field and normal owner/audit fields.
Number Length 18 with 2 decimal places caused an error	Use Length 16 and Decimal Places 2 for the corrected decimal field specification. Do not retry the rejected 18 + 2 combination.
Later Weight instructions stated Length 10	Use the established corrected decimal settings in the final schema; do not silently recreate the field.
Text location in early setup versus Geolocation in detailed schema	Use Geolocation for final Location; preserve earlier address text as legacy data if present.
Unwanted early picklist values	Deactivate legacy values after reviewing records; use the final active values in Section 3.2.
Steel Production mentioned in lookup and sharing steps	Use the Plastic Waste and Recycling Center project objects as required. No Steel Production dependency.
Tabs or roles reported as already existing	Locate and retain the existing configuration; do not create duplicates.
Order sample code used Recycled_Product__c	Use actual Order.Product__c. The customer lookup is Account__c.
Markdown links appeared inside code	Use plain identifiers such as product.Id and Trigger.new, not hyperlink syntax.
Trigger could not find processOrderStock	Save the corrected InventoryManager class before the trigger and match the method signature.
Original update list could include a product more than once	Use a map keyed by product ID for order product updates.
Original test created product without its Text Name	Provide Name in test product data.

Issue in instructions or setup	Correction or documented decision
Original tests processed orders or approvals twice	Exercise triggers once through DML; use direct handler calls only for isolated guard tests.
Three tests left highlighted skip lines	Add mixed-input guard coverage and rerun the saved class; learner reported 100 percent afterward.
New sharing rule button disabled	Allow the org-wide-default recalculation to finish and refresh.
Flow canvas looked different from reference	Accept the equivalent card layout when decision branches and actions match.

Table 29: Implementation Issues and Applied Corrections

Technical References

R1. Salesforce Trailhead, Define Sharing Rules:

https://trailhead.salesforce.com/content/learn/modules/data_security/data_security_sharing_rules

R2. Salesforce Admins, What Is a Schedule-Triggered Flow?: <https://admin.salesforce.com/blog/2023/what-is-a-schedule-triggered-flow>

8 Implementation Architecture Summary

The final application connects operational data, user access, declarative checks and Apex stock handling. Its components have the following responsibilities:

- Data Layer: five custom objects store collections, centers, products, orders and restock requests; Account supplies the customer relationship.
- Navigation Layer: custom tabs and the Re Plastic Innovations Lightning app expose the five business modules.
- Access Layer: roles, cloned profiles, sample users, private defaults and sharing rules define the documented access configuration.
- Data Quality: the stock formula displays inventory status; validation rules check order quantity and future collection dates.
- Daily Follow-Up: the schedule-triggered Flow evaluates products and creates high-priority Tasks for the low-stock outcome.
- Order Processing: the after-insert trigger reduces sufficient stock or creates a Pending shortage request.
- Restock Processing: the after-update trigger selects a transition to Approved, increases stock and invokes warehouse email notification.
- Quality Assurance: InventoryManagerTest checks core inventory behavior, with the final 100 percent coverage reported in the project completion confirmation.

8.1 Completed Feature Summary

Area	Status at documentation handoff
Custom schema, tabs and Lightning app	Built during the walkthrough; app creation explicitly confirmed.
Roles, profiles and sample users	Configured through the supplied activities; attach final assignment screenshots.
Platform 3 object access	Plastic Waste grant recorded; other four project object grants explicitly deferred.
Sharing rules, formula and validations	Configuration activities completed in sequence; attach saved settings and test evidence.
Scheduled Flow	Canvas built and activation procedure followed; scheduled Task outcome unconfirmed.
Apex inventory automation and tests	Working corrections recorded; final 100 percent coverage reported by learner.
Warehouse approval email	Helper included; actual inbox delivery not confirmed.
Production release	Not performed or claimed.

Table 30: Implementation Completion and Evidence Status

Deferred checks

- Leave the four remaining Platform 3 object grants unchanged until the learner explicitly resumes them.
- Verify Platform profile app visibility, field-level security and sample-user access; sharing rules do not replace object permissions.
- Capture an actual scheduled Task result and a real approval email before marking their end-to-end checks passed.
- Attach final saved field settings, especially geolocation/legacy fields, and the latest Apex test results.

8.2 Future Enhancements

Potential later work includes permanent approval idempotence, duplicate restock and Task prevention, stock reconciliation for order edits or cancellations, row locking for concurrent orders, required-field and negative-stock checks, explicit security review, email delivery error handling, and dashboards for collection and inventory trends.

A shortage request does not reserve the available stock or automatically fulfill the original order after replenishment. Waste-to-product conversion, processing capacity enforcement and environmental impact measurement would require additional data and business logic.

Further extensions could connect waste batches to finished products, enforce center processing capacities and add dashboards for waste collected, inventory shortages and restock turnaround. These enhancements are outside the completed implementation.

9 Conclusion

Re Plastic Innovations demonstrates a Salesforce implementation for managing plastic recycling operations and sustainable product inventory. The project establishes a structured data model for waste collection, recycling centers, recycled products, customer orders and restock requests, and exposes those records through a dedicated Lightning application.

The stock formula and validation rules support understandable inventory status and valid data entry. Apex connects new orders to inventory deductions when stock is sufficient and creates shortage requests when it is not. Restock approval increases inventory, while the warehouse email helper and daily low-stock Flow provide follow-up mechanisms.

The work developed practical skills in Salesforce configuration, data modeling, relationships, access administration, Flow Builder, Apex classes and triggers, and automated testing. The learner reported completion of the Apex testing activity with 100 percent coverage. Final screenshots provide the evidence for the saved setup and the tested results.

The documented completion boundary remains explicit: four Platform 3 object grants were deferred, and actual scheduled Task creation, email delivery and complete user-access behavior still require confirmation. No production deployment or automated fulfillment of insufficient-stock orders is claimed. These limits preserve a clear distinction between the implemented learning solution and possible future operational enhancements.